

CodeInsight: A BEHAVIORAL ANALYTICS-DRIVEN
WEB SYSTEM FOR CONCEPT-LEVEL
DIFFICULTY DETECTION AND EARLY INTERVENTION
IN INTRODUCTORY PROGRAMMING COURSES
IN PAMPANGA STATE UNIVERSITY

A Capstone
Presented to the Faculty of
College of Computing Studies
Pampanga State University in
Partial Fulfillment of the
Requirements for the Degree
Bachelor of Science in Information Technology

By:
Nunag, Nathaniel H.
Hizer, Rosmer D.
Cortez, Angela A.
Estanislao, Yngwie Djinn M.
Lugtu, Nico C.
Nucum, Kerby V.
Tizon, Sean Aaron M.
Tolentino, Paul Andrei P.

April 2026

TABLE OF CONTENTS

Chapter I – The Problem and Its Background

Introduction.....	
Purpose and Description.....	
Objectives of the Study.....	
Conceptual Framework.....	
Scope and Limitations.....	

Chapter II - Review of Related Literatures and Studies

Related Literature.....	
Related Studies.....	
Technical Background.....	

Chapter III – Methodology

Research Design.....	
Software Development Model and Phases.....	
Requirement Analysis and Documentation.....	
Research Tool and Instrument.....	
Statistical Treatment of the Data.....	
Respondents of the Study.....	
Sampling Technique.....	
Implementation Plan & Result.....	
Ethical Consideration.....	

CHAPTER 1

Chapter 1 presents the foundational context of the study, including the background and rationale for developing CodeInsight, a behavioral analytics-driven web system for detecting concept-level difficulties in introductory programming courses. It outlines the purpose and description of the system, the general and specific objectives, and the conceptual framework guiding the research. The chapter also discusses the significance of the study for stakeholders such as the university, instructors, students, and future researchers, along with the scope, limitations, and key definitions that delimit the investigation.

Introduction

With technology becoming a huge part of everyday life, knowing how to program has become one of the most important skills a person can have in today's generation. Because of this, governments, schools, and different industries have started to realize how important computer education is in preparing students for a world that is becoming more and more dependent on technology. As a result, more computing programs have been established and a growing number of students have been enrolling in programming-related courses (National Academies of Sciences, Engineering, and Medicine, 2021). At the forefront of these programs are introductory programming courses, which form the basis of all advanced computing knowledge. These classes help the novice programmer understand basic principles in programming including variables, conditions, loops, functions, arrays, and object-oriented programming — which are the essential elements of programming. Given this foundational role, as the need for computing graduates continues to rise worldwide and in the Philippines, institutions such as Pampanga State University, which is ranked as the first higher education institution in Central Luzon (uniRank, 2025), are at the forefront of producing computing graduates, and thus, the effectiveness of introductory programming education is of utmost importance.

However, there has been a consistent pattern that introductory programming courses have one of the highest failure rates among all other educational courses. This problem can be seen in the Philippine context, wherein early identification of students at risk in first-year programming classes has been regarded as a priority (Bibangco & Dionson, 2024). Rahimi et al. (2025) further supported this, noting that the struggles of students in introductory programming classes have continued to result in high failure rates in recent years. Beyond the general failure rates, beginner programmers also face difficulties with particular concepts including loops, conditionals, and object-oriented programming. When these challenges are left unidentified or unresolved at an early stage, they tend to develop into more serious obstacles for students throughout the duration of the course. A systematic review of the literature on teaching and learning introductory programming in higher education institutions revealed that educators across the world have a problem identifying which students are struggling with particular concepts early enough to make a difference (Medeiros et al., 2019). The recurrence of this challenge across different countries and different years indicates that this is not a problem of student or student instructor quality, but a systemic problem with the mechanisms in place for detecting and addressing programming difficulties.

Several studies have sought to address the challenge of student difficulty in introductory programming courses through technology-based interventions. Among the most significant contributions in this area is the use of learning analytics to monitor student behavior and forecast academic outcomes. Leinonen et al. (2022) demonstrated that behavioral metrics derived directly from students' programming activity specifically time spent on task, number of errors, and number of submission attempts are among the most reliable predictors of student performance in introductory programming courses, with fine-grained behavioral data showing stronger correlations with actual course outcomes than broader, grade-level measures. This finding was further supported by Namoun and Alshantiti (2021), who confirmed that behavioral variables collected during the learning process are effective predictors of student academic performance in online learning

environments. As far as the local setting for 2019s, there have been instances where learning management systems have been deployed in the teaching of programming courses within Philippine higher education institutions, with such LMSes having capabilities like automatic grading, assignment submissions, and even tracking (Dela Rosa, 2023). It is clear from such research that computerized tracking and analysis have the potential to play an important role in improving the results from programming education. Such practices form part of a wider global trend, whereby student behavior data acquired through learning is utilized for decision-making purposes.

Despite these developments, current learning analytics systems fail to achieve the concept-level granularity required by educators to intervene successfully. The majority of learning analytics solutions currently work at the performance level; they identify learners who are at risk of failing but cannot determine which particular programming concept is giving students trouble (Medeiros et al., 2019). While contemporary web-based tools for programming courses can follow assignment and grade completion levels, they cannot collect behavior-level information related to the process of programming, like the number of retries that a learner performed when trying to solve a particular task or the duration of working on certain programming concepts (Namoun & Alshanqiti, 2021). Not long ago, Paolucci et al. (2024) emphasized the significance of concept-level difficulty assessment and the possible implications of this issue for pedagogical practices, suggesting that learning analytics not only concentrate on passing/ failing students but also be able to recognize problems at the concept level.

Specifically, the core problem is that instructors of introductory programming courses currently lack a system that can translate student behavioral coding data into actionable, concept-level difficulty indicators in real time. Although early warning systems can alert teachers that a particular student is having difficulties, they do not identify which programming principle is responsible for these difficulties, making it impossible for teachers to gain enough insight into where the problem lies in order to

intervene and offer assistance. If no action is taken to rectify this issue, students that have difficulties understanding basic concepts in their introductory programming classes will find themselves struggling further until they either fail the course or leave college altogether, an outcome that has been proven by various studies conducted on this topic (Medeiros et al., 2019; Rahimi et al., 2025).

To solve the identified challenge, this research suggests developing and designing a programming concept difficulty analytics system called CodeInsight for introductory programming courses. CodeInsight aims to systematically record the student's behavioral programming data using CDS formula, including programming errors, retry attempts, and time spent on the activity, and correlate them to individual programming concepts involved while solving the exercise (Leinonen et al., 2023). Instructors are presented with real-time difficulty assessments related to individual programming concepts via a dashboard within the CodeInsight analytics system, which enables timely interventions before student difficulties evolve into problematic learning experiences. Alternatively, CodeInsight provides an online platform where learners can engage in coding activities and obtain immediate feedback on the structure of their codes. CodeInsight aims to fill the gap between the occurrence of student difficulties and the intervention by instructors through translating student behavior into levels of complexity in relation to different programming concepts. This research is pertinent to certain key stakeholders in the educational sector. From an educator's point of view, difficulty analytics at the level of concepts would mean transitioning from a reactive, grades-focused approach towards instruction to a more proactive approach which involves analyzing real student behavior and coding experience to come up with appropriate interventions. For students, knowing the reasons behind their poor performance will enable them to reflect on what they are doing and improve accordingly rather than being aware of the poor performance in general without knowing where the weakness lies. The CodeInsight system is very relevant to the Philippines higher education system.

As regards the research literature, this study adds value by contributing towards the expanding field of learning analytics for programming education. This field can be recognized as one of those fields which have received little attention from scholars conducting systematic reviews (Medeiros et al., 2019; Paolucci et al., 2024). Also, the implementation of this system can be conducted within the time and budget constraints of this research project because of its web-based nature.

Purpose and Description

The general purpose of CodeInsight: A Web-Based Programming Concept Difficulty Analytics System for Monitoring and Supporting Beginner Programmers in Introductory Programming Courses at Pampanga State University is to monitor and support beginner programmers enrolled in introductory programming courses of Pampanga State University. This web-based system for programming education has the following attributes:

User Login and Registration Users can create and access their accounts depending on the roles they have in the system.

1. **Instructor Login** – This login is for teachers to create and manage programming exercises, view analytics dashboard, track the difficulty of concepts for each student and the whole class, get early warning alerts and create long-term progress reports.
2. **Student Login** - This login lets students use the built-in code editor to access and answer programming exercises, see their own concept difficulty profile, and keep track of their progress over time.

The editor distinguishes between two submission modes. The Run button tests the student's code against the first visible test case only, provides immediate feedback, and does not save the attempt to the database or count it toward any behavioral metric. The Submit button tests the code against all test cases including hidden ones, saves the attempt to the database, and counts it in the CDS computation, ensuring that only intentional, official attempts are measured.

Exercise Management. The system lets teachers make programming exercises and tag each one with one or more programming concepts, like variables, conditionals, loops, functions, arrays, and object-oriented programming. This makes sure that all the behavioral data collected during student activity is relevant to the concepts being taught.

Web-based Code Editor. Students can do programming exercises right in the system using a built-in code editor. All coding done in the editor is automatically recorded in real time, along with the following behavior indicators:

1. Error Frequency – the number of mistakes a student makes while doing an exercise.
2. Retry Count – the number of times a student tries an exercise again or sends in a solution again.
3. Time-on-Task - the total amount of time a student spends on an exercise.

Code Structure Verification. CodeInsight checks that the submitted code shows real conceptual engagement with the tagged programming construct before the Concept Difficulty Score engine processes the submission. This verification uses treesitter, an incremental parsing library that creates an Abstract Syntax Tree (AST) of the submitted C++ code (Chung et al., 2022 and Sun et al., 2023). It also uses canonicalized pattern matching from IBM's Project CodeNet dataset. The verification is completely deterministic and based on rules; it does not use machine learning. The following checks are performed, in order:

1. **Construct Presence Check** - checks that the AST has the node type that is associated with the tagged concept (e.g. a `for_statement` or `while_statement` node (for a Loops exercise).
2. **Non-Empty Body Check** - This checks that the detected construct has at least one meaningful statement in its body, rather than being an empty shell.
3. **Variable Usage Check** - This checks that the construct uses variables instead of just constants which makes sure that logic, not hardcoded values, controls how the program works.

4. **Output Dependency Check** - checks that the construct really does affect the program's output, making sure that it is not just there but not doing anything.
5. **Known Bad Pattern Matching** - compares the submission's AST to a library of common beginner mistakes taken from Project CodeNet (e.g., `sum = x` instead of `sum += x` for Loops; function definition without a return statement for Functions; subscript with an out-of-bounds literal index for Arrays). Submissions that match known wrong patterns are marked and not counted in the CDS.

If a submission fails any part of this verification, it is marked as `structure_valid = false`, not counted toward the Concept Difficulty Score, and shown in the Structure Violations panel of the analytics dashboard for the instructor to review.

Concept Difficulty Score (CDS) Engine. The system uses a behavioral formula-based difficulty mapping engine to turn each student's recorded coding activity into a Concept Difficulty Score (CDS) for each programming concept. The CDS uses a clear, citation-based formula $CDS = (0.40 \times NER) + (0.35 \times NRS) + (0.25 \times NTS)$ to calculate difficulty from error frequency, retry count, and time-on-task. This gives each student a clear and measurable index of concept-level difficulty that is updated in real time after each official submission.

1. **Analytics Dashboard.** The system provides teachers with a holistic view of the difficulty level of the course for their students both at the individual and group levels. This includes:
2. **Difficulty Heatmap** – A matrix that displays the CDS of each student in the class for every programming concept, enabling teachers to quickly identify areas where most students are experiencing difficulties.
3. **Early Warning Alerts** – Automated alerts that inform teachers when a student's CDS for a particular concept crosses a threshold to enable intervention before the student gets low marks.

4. **Individual Student Profile** – An in-depth assessment of a single student's CDS across all concepts, identifying the areas that need the most attention.
5. **Longitudinal Progress Reports** – are reports that show how a student's or class's concept difficulty changes over time during the course.
6. **Deterministic Micro-Concept Difficulty Analysis** -
These are system generated teaching suggestions that use rule-based analysis of the following signals to find the exact sub-concept that is confusing a student. It then makes a specific teaching suggestion for each Early Warning Alert:
 - a. Compiler error patterns (GCC) are the kinds and number of errors that happen when the student's code is compiled and run.
 - b. Code structure (AST nodes present/absent) – whether the required programming constructs are present, correctly formed, and semantically used as determined by tree-sitter AST parsing.
 - c. Test case failure patterns (boundary cases, specific inputs) – shows which test cases the student's submission fails, including boundary and edge-case inputs. This shows where the logic goes wrong.
7. **Class-Wide Micro-Concept Difficulty Report** - puts all of these insights together into a single pattern analysis for the whole class. This report helps the teacher decide what to do next in the lesson.
8. **Structure Violation Report** - this is a list of all the submissions where the required programming construct was missing, empty, or used incorrectly. It shows the student, the concept, and the specific validation message for the teacher to look over.
9. **Academic Integrity Flags** - a panel that shows all the integrity alerts the system has sent out, such as hardcoding, behavioral, passive behavioral and blank only flags, so that the instructor can review them and take the right action.
10. **Academic Integrity Monitoring** - An academic integrity monitoring layer works alongside the CDS engine to make sure that difficulty scores are

accurate by finding and flagging fake submissions for the teacher to look at. This layer contains the following mechanisms:

- a. **Hardcoding Detection (literals in output instead of variables)** - detects submissions that produce the correct output by mapping some inputs to outputs instead of using an algorithm to compute the output. Submissions detected are not considered for class normalization so as not to skew other students' CDS scores.
- b. **Blank/Template-Only Submission Detection (code unchanged from starter)** - finds submissions where the student hasn't made any real changes to the starter code, which means they didn't really try the exercise. These submissions are marked and left out of CDS computation so that difficulty scores don't go up.
- c. **Behavioral Anomaly Detection (unusual patterns: instant success, extreme speed)** - flags statistically unlikely performance patterns, like a student who has always had a hard time but suddenly submits a perfect solution on the first try, or a student who finishes an exercise in a time that is much shorter than what is normal for the class, sending an integrity alert to the instructor for consideration.
- d. **Code Growth Anomaly Detection** - monitors line count trajectory across a student's submission history. In this system, a growth spike exceeding 30% between consecutive submissions is treated as a potential indicator of bulk code insertion rather than incremental debugging.
- e. **Passive Behavioral Logging** - keeps track of how many times you switch between browser tabs and paste things during a coding session and shows these as contextual indicators on the Academic Integrity panel. These are just suggestions; all integrity-related decisions remain at the discretion of the instructor.

11. **Concept Tagging.** When creating an exercise, teachers can give it one or more programming concept tags. These tags are the semantic backbone of the

analytics engine. They ensure that all behavioral data collected is relevant to the right concept and presented correctly in the CDS.

12. **Progress Tracking.** The system provides students with their own concept difficulty profiles. This is useful for users to know which programming concepts are the most difficult for them and to self-reflect on how to improve in the course.

Objective of the Study

This section presents the general and specific goals that the study seeks to accomplish. These objectives serve as the guiding framework for the design, development, and evaluation of CodeInsight, and consequently direct the course of data collection, system implementation, and analysis throughout the entire research.

General Objective

The general objective of this study is to design and develop CodeInsight, a WebBased Programming Concept Difficulty Analytics System intended to monitor and support beginner programmers enrolled in introductory programming courses at Pampanga State University.

Specific Objectives

Specifically, this study aims:

1. **To design and develop a web-based system that enables instructors to create programming exercises manually tagged to specific C++ fundamental programming concepts**, thereby mapping difficulty at the concept level through a transparent, citation grounded behavioral formula reinforced by academic integrity safeguards that ensure all computed difficulty scores reflect genuine learning attempts rather than shortcuts or circumvented submissions.

2. **To implement a Concept Difficulty Score analytics engine that captures behavioral coding data, specifically error frequency, retry count, and time on task**, thus enabling proactive identification of struggling students before poor academic performance becomes evident, with code structure verification in place to prevent hardcoded, memorized, and template only submissions from artificially distorting scoring accuracy.

3. **To develop an analytics dashboard presenting class wide difficulty heatmaps, individual student difficulty profiles, early warning alerts, longitudinal progress monitoring reports, and rule based micro concept difficulty insights** derived from deterministic analysis of compiler error patterns, Abstract Syntax Tree structures, and test case failure modes, which collectively translate raw behavioral data into actionable teaching recommendations without requiring any external service or machine learning dependency.

4. To evaluate the quality of CodeInsight based on selected characteristics defined under ISO/IEC 25010:2011, specifically Functional Suitability, Usability, and Performance Efficiency, using the System Usability Scale and pre and post assessment instruments administered within an introductory C++ programming course.

Significance of the Study

Programming subjects are considered as one of the most difficult subjects for beginners. The inability to pinpoint and comprehend the exact nature of the problem faced by the students has been one of the common issues faced in the field of programming. These systems usually depend on the final results of the students' performances or the grade received, which is more reactive than proactive in providing insights into their difficulties.

This research project attempts to solve this issue by creating a concept difficulty analytical tool for learning programming called CodeInsight. It is a

web-based system that can give instructors insight into their students' performances and help them rectify their problems in advance.

The following groups are expected to benefit from this study:

Pampanga State University – College of Computing Studies (PSU CCS)

CodeInsight offers the university a system with means of monitoring problems faced by students in learning programming courses at the concept level. This helps the university directly address one of its most persistent academic challenges, high failure and dropout rates in computing programs.

Although other methods require the use of licensed software or APIs from other companies, Codeinsight does not have additional expenses for each submission. This allows the university to maintain and sustain a system that is independent of any organizational affiliation. It also supports academic integrity by detecting and discouraging learning shortcuts, and by providing a working system that supports better student learning over time.

Instructors

CodeInsight has moved away from the conventional monitoring based on grade results to a proactive early warning based on concepts. This means that when the instructors see that there is an issue to a student code, it can be identified down to the specific micro-concept mistakes being made, like an off-by-one loop mistake instead of just noting that the concept itself is hard.

Allowing instructors to implement appropriate measures without resorting to tedious code analysis by hand. In addition, academic Integrity features enable teachers to rely on accurate and truthful data.

Students

By giving students more transparent and actionable feedback about their code, CodeInsight is capable of aiding in effective learning. While most programs would simply output something like “test failed,” with CodeInsight, students receive deterministic, rule-based feedback which directs their attention to specific problems they face.

As a result, learners can concentrate only on necessary areas of work, which promotes self-regulated learning and lowers chances of making the same mistakes again and again. Additionally, hidden test cases and structure analysis prevent learners from using hardcoding techniques and rote memorization to complete assignments.

Future Researchers

This research will contribute to the programming learning analytics by introducing a deterministic concept level difficulty measurement methodology. By doing so, this project gives researchers a practical example of how rule-based microconcept analysis can provide beneficial results for the field. The inclusion of measures to ensure academic integrity into behavior analytics also solves an important problem in measuring difficulty of programming concepts. Thus, the findings and methods used during this research will help future studies in expanding micro-concept analysis and developing difficulty modeling.

Scope and Limitations

CodeInsight will be developed and tested as a web-based software that can detect and assess programming concepts complexity among first-year beginner programming students enrolled in introductory programming classes at PSU College of Computing Studies for the academic year 2025-2026. In an introductory C++ class with the seven concepts, Datatypes, Variables, Conditionals, Loops, Functions, Arrays, and OOP, this web application will automatically track the learning trends, behaviors, and concepts complexity. Behavioral data that includes the number of

errors, number of retries, time-on-task, and number of submissions will only be recorded from students' interaction on the software for the entire semester and old quizzes/coding problems for the computation of relevant metrics. A direct GCC C++ compiler, executed either on the local or on the cloud server, (Oracle Cloud free tier, or similar) compiles and executes the students' submission. With the use of Abstract Syntax Tree (AST) from tree-sitter, the structure of the students' codes are verified deterministically with matching patterns from IBM Project CodeNet (around 15,000 beginner C++ solutions). After that, it determines that required programming constructs, non-trivial bodies of these constructs, and the use of programming constructs as manipulation tools of output are already included. The system infers microconcept complexity based on over 14 rules for each of the seven concepts using trends of errors and AST detection, test case validation, and source code analysis. Five deterministic measures for Academic Integrity: detection of hardcoding, blank or template codes, massive growth, and behavioral irregularities will measure whether or not CDS accurately reflect real learning attempts. Analytics are then generated as an instructor dashboard with overall class complexity heatmap, notifications for CDS, student's personal learning curve on how students progress over time, and rules based microconcept analysis. These aspects will be evaluated against selected characteristics in ISO/IEC 25010 Function Suitability, Usability, and Performance Efficiency through System Usability Scale (SUS) and before-and-after pre-/post-assessment tests

There are numerous limitations within this study. Firstly, the validity of CDS depends on the instructor's ability to correctly classify each exercise by appropriate programming concept, otherwise there can be falsely estimated difficulty values due to any form of error. Secondly, CDS can only measure external, observed behaviors (errors, retries and time) as opposed to psychological components (confusion) and affect related concepts (anxiety and motivation). Third, the AST based structure checker is deterministic, thus it may be unable to identify complex semantic loopholes, or solutions which are technically logically sound but unorthodox. Fourth, the microconcept rule set has only been applied to a limited number of common

mistake-inducing patterns (14+) during program generation; non-common mistakes are therefore unclassifiable with specific microconcepts. Fifth, the hard-coding detector only identifies specific types of common patterns (e.g. Static string values within the printf function); more intricate techniques would not be detected, though unlikely at introductory level. Sixth, there is no examination of pairwise similarities between different student solutions, as plagiarism detection is not included. Seventh, the system makes no judgment on algorithmic creativity, efficiency, correctness of code past test cases, or style of the program written. Eighth, the system will be tested over only a single semester, so no data can be compiled about the long-term impact on pass rates. Ninth, the teacher-relative system requires at least 3 submissions from a class to ensure the validity of the normalization, and a class size of very small numbers will yield an unreliable classification. Tenth, it is not a full learning system – there are no provisions for grades, course materials, messaging, multimedia feedback, etc., and student comments on solutions are only readable. Eleventh, results are highly specific to PSU CCS, first-year students who are enrolled to C++ introductory courses. It would be unreliable to extrapolate findings to any other demographic, level, language or institution. Twelfth, no data is collected about instructor or student opinions (besides the usability questionnaire).

Conceptual Framework

The inputs, processing, and outputs required during the development phase can be seen in the CodeInsight Programming Concept Difficulty Analytics System Framework. The above-mentioned framework offers a roadmap to the construction of the system that will be used in analyzing programming ability.

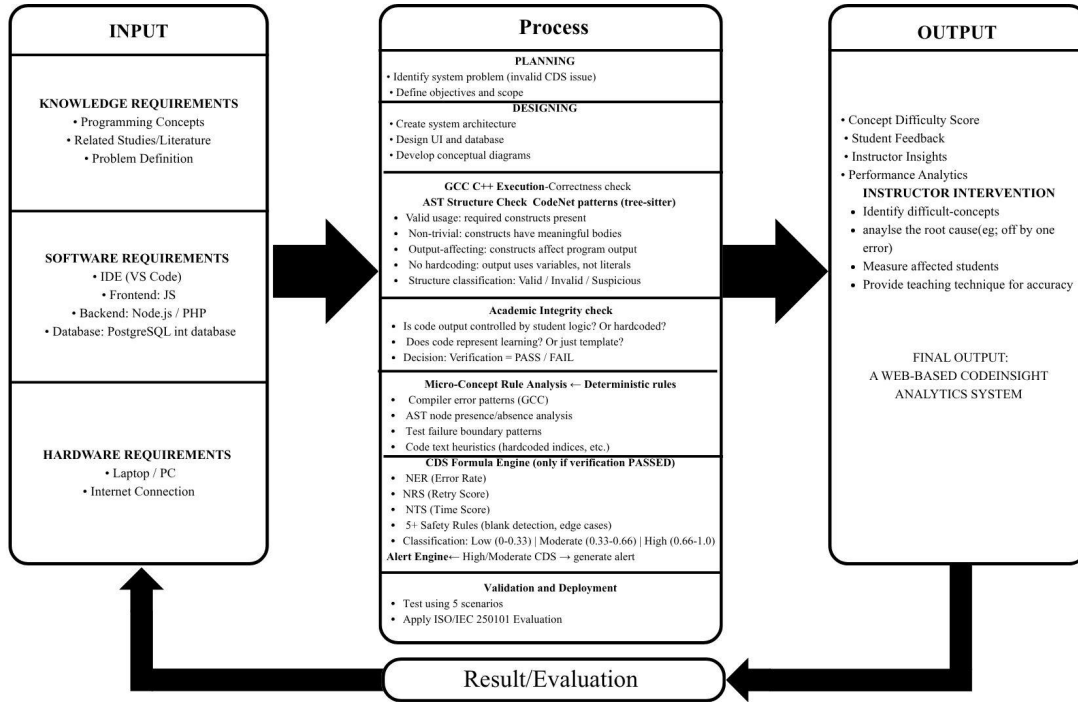


Figure 1. Conceptual Framework

Inputs include data collection, research materials, and resources required for the development of the system. These encompass the definition of tasks, example student code inputs, and fundamental programming constructs, such as loops, conditional statements, and arrays. Also included are relevant literature and research studies that provide support for the construction of the system. The hardware and software resources required include development software, database systems, and code analysis software.

Phase of Process comes next, as per Software Development Life Cycle (SDLC). The process starts off by recognizing the main issue; that is, wrongly judging the difficulty level of students due to wrong usage of the required concepts of programming. Once the goals have been established, then comes the phase of Data

Gathering through sampling of codes and creating test cases based on various programming scenarios.

One of the core parts of this model is called Code Processing and Analysis Pipeline. As part of this stage, a compiler (for example, GCC for C++) is used to execute the code written by a student. Further, an analysis of Abstract Syntax Tree (AST) structure is performed using tools such as tree-sitter as well as coding patterns in order to identify whether needed programming constructs are used appropriately and make contribution to the output of the program. Also, inappropriate solutions can be found, for instance, hardcoded ones.

Afterward, Micro-Concept Rule Analysis should take place. This type of analysis uses rule-based techniques to detect compiler errors, missing/incorrect code structures, failed test cases, and typical programming mistakes like off-by-one and incorrect conditions.

In order to validate code analysis results and determine their reliability, the Academic Integrity Check must be completed. During this process, it will be determined whether the output of the program is governed by the student's logic or just a hardcoded solution to meet certain test cases. In addition, it will be evaluated whether code analysis reflects real knowledge of the student or represents just a copy-pasted solution from a template program. Verification result PASS/FAIL is assigned as a consequence.

If the code passes verification, the source code will be analyzed by the Concept Difficulty Score (CDS) Computation Engine. This process involves computing difficulty experienced by students using several measures including Error Rate (NER), Retry Score (NRS), and Time Score (NTS) among others. Edge cases validation rules are also used to validate computed results that fall into the categories of low, moderate, and high difficulty.

For students that have been categorized as moderate or high difficulties, the system triggers an Alert Generation mechanism to generate alerts for students' difficulty levels. Alerts are sent to the Analytics Dashboard which visualizes performance of each student in the course using heat maps, student profiles, and class level reports.

Lastly, the system allows for Instructor Intervention where teachers can use analytics provided by the system to make sound decisions about difficult concepts, why the students find them difficult, how many have difficulties with them, and what kind of teaching methods should be employed. In order to assess the quality of the system, it has been tested according to the ISO/IEC 25010 standard criteria for evaluating software such as functionality, reliability, usability, performance efficiency, security, maintainability, and portability using various test cases.

The output phase of the framework is the full-fledged CodeInsight Programming Concept Difficulty Analytics System implemented in a web-based environment that can generate Concept Difficulty Scores, provide automatic feedback, and give insights from the analysis performed using its analytics dashboard.

On the whole, the CodeInsight framework brings improvements to programming education because of improved and fair assessment, elimination of invalid assessments, informed decision-making, efficient feedback delivery, faster assessment procedures, and overall enhanced learning experiences.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter discusses the studies and literature that serve as the conceptual basis of CodeInsight: A Behavioral Analytics-Driven Web System for Concept-Level Difficulty Detection and Early Intervention in Introductory Programming Courses. These are the concepts, theories, models, and systems that pertain to programming, learning, behavior markers, analytics, and test methods for the purpose of identifying and addressing concept-level difficulties in introductory programming courses. Moreover, the previous studies are analyzed as to their strengths, weaknesses, and gaps in research, which will serve as the reason why the proposed system is needed at this point in time.

Related Literature

It has always been true that there are high rates of failure and dropouts in introduction to programming classes all over the world. As pointed out by Batten (2025), although there has been a steady increase in computing enrollments by 9.9%, which means that the increase in students enrolled in the field of computer science, computer engineering, and information resulted in computing enrollment being 6.8%, success in introduction to programming is not easy. According to Pires (2024), the issue of student failure in basic programming classes persists globally. Additionally, Alvarez (2023) indicates that after conducting research for four years involving 959 first-year engineering students, 19.3% passed the course while they were freshmen, and 25.0% dropped out of the institution, meaning that roughly one-third of the students failed to complete introductory programming courses. In most cases, this problem arises from two major causes, including: (1) process-based issues because of the lack of knowledge about syntax; and (2) concept-based issues since misconception is caused by logic sequencing, data structures, and control structure. This perception is also supported by Bigotte de Almeida et al. (2023), who argue that

lack of mathematical and logical thinking understanding and skills may lead to poor comprehension.

Learning analytics (LA) applied to programming education is a promising direction of research allowing tracking students' actions and predicting their outcomes. In a systematic review based on 85 other studies, As stated by Omer et al. (2023), among the common metrics used in LA include frequency of compiling, time spent submitting, and passing test cases percentages, along with an absence of metrics that solve issues related to specific concepts and inconsistency with theory, as there are many lecturers who fail to work with analytics tools effectively. According to the authors, an effective LA system should provide interpretable and immediate feedback and can only be applied to beginners in programming. López et al. (2021) showed that process-related and product-related metrics were helpful for measuring success in learning activities. Based on their use of sequence analysis and process mining techniques on data collected from 250 students, they developed categories of behavior such as systematic explorers, trial-and-error coders, and disengaged repeaters, with the claim that the adoption of singlefactor measures like test scores is likely to conceal behavioral tendencies that precede learning problems. Moreover, using entropy and community detection as metrics, According to the research conducted by Mai et al. (2023), in relation to entropy as the predictor of unpredictability of students' behavior during the editing process, it was found that there is a direct correlation between the high entropy levels and poor outcomes for students, and moreover, machine learning models can identify the problem faced by students within two weeks of training.

The Abstract Syntax Tree (AST) technique has proven to be extremely effective because, on the one hand, it is capable of assessing the syntactic correctness of students' code and, on the other hand, of analyzing the logical structure. In contrast to output testing, which merely confirms whether the result achieved by the program corresponds to the required one, AST analysis allows looking more closely into the program code and checking whether students were able to implement a necessary idea

in their code. It becomes highly significant in the scenario where the student is being taught how to perform basic programming, and he/she may be able to design a program that outputs correctly but does not make use of the desired algorithm. As mentioned by Chung et al. (2022), the integration of AST analysis along with the knowledge graph is extremely useful in enhancing the accuracy of the evaluation process. Furthermore, Sun et al. (2023) have stated that the use of AST frameworks enables the retention of syntactic and semantic features of programming, making it suitable for educational technology applications.

Learning progressions are models designed to outline the process of development of students' knowledge in a certain order as well as to diagnose a student's level in terms of the process of learning. Specifically, according to Alonzo (2022), teachers involved in working with learning progressions transform the way they assess a student's knowledge, from merely marking an answer as right or wrong to identifying the exact place of the student within the learning progression of concepts. More than that, according to Alonzo, a correct answer does not imply conceptual understanding. As cited in Rodríguez-Mora et al. (2024) research a systematic review based on the PRISMA framework, misinterpretations may arise due to existing knowledge, faulty teaching, and the abstractness of the idea; however, finding out the misinterpretation was not enough, since the misinterpretation should have been associated with a particular phase of the learning process. Through their meta-analysis of 218 papers, involving more than 18,000 participants altogether, Pacaci et al. (2024) found that conceptual change interventions like cognitive conflict, cognitive bridging, and ontological category shifting were extremely effective. These studies highlight a problem that remains unresolved: There is currently no comprehensive model that maps particular misconceptions to particular learning phases and recommends suitable instructional approaches for each, something that is what CodeInsight's concept-level difficulty scoring is meant to solve.

Formative assessment is considered one of the best teaching strategies, especially in enhancing student performance, such as learning programming subjects, where the development of knowledge is carried out step by step. According to an umbrella review conducted by Sortwell et al. (2024), based on the evaluation of outcomes of 13 meta-analyses, it is evident that formative assessments prove to be efficient in regard to their impact on the results of academic work of pupils, which means that the suggested approach is considered legitimate when implemented during programming courses. Employing Formative assessments has been reported to have a significant effect on the academic outcomes of students (level of impact = 0.72), according to Karaman (2021). In another study conducted by Rouben et al. (2023), the instant feedback in the framework of formative assessment has allowed instructors to identify mistakes made by students in their reasoning, that help them understand the learning material clearly.

The Concept Difficulty Score (CDS) is calculated through this formula: $CDS = (0.40 \times NER) + (0.35 \times NRS) + (0.25 \times NTS)$, using these three combined metrics that are grounded by these three studies. The first study, NER or the Normalized Error rate that has the highest weight of 0.40, is based on the study of Altadmri and Brown (2015) which shows that error frequency is the greatest predictor for difficulty in programming language. Secondly, the Normalized Time score or the NTS weighted 0.35, is based on the discovery of Ahadi et al. (2016) that the formula for calculating the parameter involves the ratio of the number of attempts made towards getting the right answer and the maximum number of attempts possible. The foundation of the said parameter stems from the findings of Ahadi et al. (2016) that attempts made by the students in solving programming-related problems are a better predictor than correctness. Lastly the Normalized Time Score which has the weakest weight of 0.25 that supports the study of Leinonen et al. (2021) whose research shows that very detailed information regarding the time taken to work on a problem provides a reliable secondary indicator of programming difficulty in conjunction with error and retry data.

CodeInsight operationalizes these formative assessment principles through its Concept Difficulty Score (CDS) in which is deployed in CodeInsight for learning analytics as well as automated feedback this lead to generation in programming assignments within the system when analyzing through actions taken by the learner for each of the micro-concepts. is done in relation to a specific programming concept (Banihashem et al., 2025; Hao et al., 2021). Simply put, CDS is not involved in providing data for analysis, rather, it acts as an automatic intervention mechanism in cases where the learner's understanding of the programming concept is too difficult and crosses the threshold set. Moreover, CDS also offers suggestions on possible interventions that would assist instructors in handling the learners' difficulties effectively. More importantly, the use of AST-based structural validation and automated code assessment mechanisms increases the effectiveness of the analysis because it ensures that learners cannot complete the assigned task through the use of misleading codes (Barczak et al., 2023; Messer et al., 2023).

The longitudinal research conducted to analyze the skills of the students involved in programming has shown to be much more effective than the one-time analysis of skill development of the students. According to Gao et al. (2025), it has already been evaluated what kind of learning trajectories students who attend introductory programming lessons demonstrate in terms of their engagement and success in exams and tasks. It has been revealed that various trajectories exist; however, some of them show continuous growth while others demonstrate erratic growth. Likewise, Yao and Lin (2025) have performed a long-term analysis of programming competence-based education and have established that such an approach helps develop personalized teaching approaches and identify various learning trajectories associated with programming skills, computational thinking, and problem-solving. Hsu and Gainsburg (2021) have confirmed the benefits of a longitudinal analysis of programming development and demonstrated that such an

analysis provides a chance to detect learning trajectories, including those of growth and decline, which could not be observed in a single evaluation.

Plagiarism and hardcoding are the two main forms of academic integrity violations that plague the field of programming education and pose validity issues both in terms of learning processes and skills. Specifically, according to Senger et al. (2022), around 30% of novice programmers' assignments contain instances of plagiarism and too simple coding, which means that many students fail to engage in programming practice seriously and consequently prevent themselves from proper skill evaluation. What is more problematic about the issue today is that technology develops fast, including large language models. It must be mentioned that the methods commonly used by educational institutions to detect coding-based academic misconduct are structural similarity hashing, code matching software, and machine learning-based vectorization. However, each of the described methods may become resource-intensive and unmanageable. Academic integrity issues have been addressed within the scope of the design of CodeInsight via behavior analysis to detect cases of hardcoding. CodeInsight investigates collection of personal data under Australia's Privacy Act from 1988. It has issues with being open and honest. Examples and digital issues in the investigation demonstrate this prior to considering proportionality, it suggests a two-phase test to ensure transparency and fairness. This is based on cases in courts and principles in constitutions. The idea is to strike a balance between what people want for their information and how big data is used. The investigation intends to make the data practices easier with people's privacy in mind through the use of personal data and the Privacy Act 1988 (Gregorczyk, H., 2023). CodeInsight is used for finding privacy issues for review. It doesn't say what users have done. Instead, it allows teachers to make the decision. This way teachers can make assessments based on the situation. CodeInsight marks problems for review. Then teachers examine these problems. They get to choose what to do. This helps to ensure assessments.

Related Studies

A system developed by Utamachant et al. (2023), a learning analytics platform that supports iterative learning intervention in programming courses called i-Ntervene. The system features include integrating an LMS with an automatic code grader and analyzing learning behaviors from activity gaps, such as class engagement, class attendance, and assignment practice. Additionally, it can evaluate subject understanding level using a 3-level decision tree and provide concept-based programming tutorials and visualizations to support instructors in early intervention decisions. The study was evaluated by 253 first-year university students using a Java programming course, which concluded that interventions for at-risk students must take a particular approach. However, despite its success, the system's limitations were limited only to assignment scores; tutorial sessions were found effective only for conceptual and structural topics, and could not detect syntax errors made by students in external IDEs.

A study conducted by Mai et al. (2022) examined learning behavior and academic performance among 400 first-year students by analyzing event logs. It was found that early behavioral data could predict student performance before the final assessment by outlining the activities students used, such as practice tasks, lab sheets, and lecture access. However, it provides only a general overview of the course and cannot identify which programming concepts students were struggling with. Similarly, research led by Maranga et al. (2024) focused on adding a behavioral assessment feature to CodeChum, a programming platform. The feature focuses on behavioral student performance indicators, such as the time spent and the number of executions for each submission. However, it focused only on adjusting problem difficulty and could not provide an analysis of the specific programming concepts where students struggled.

According to research by Dela Rosa (2023), the researcher examined the use of Canvas LMS in a programming course and found that its web-based platform was well-suited for programming activities among BSIT students. While Canvas LMS

represents a viable tool in programming education, it could not be determined which specific programming concepts caused the difficulty. A system developed by Pak et al. (2022) is a built-in, web-based IDE that can track coding behavior, errors, and submissions. The system provided automated error guidance to help students learn from their coding mistakes. However, the system fails to provide difficulty level information for each programming concept.

Despite their contributions, these studies share a common limitation: each reviewed system operates at a level of generality that does not fully support effective instructor intervention. None of the five studies presents a system capable of automatically diagnosing which programming concept is difficult, identifying the responsible sub-concept, and delivering this information to instructors in an actionable format. Additionally, none of the reviewed systems addresses whether behavioral scores accurately reflect learning attempts. For instance, if a student circumvents problem-solving by submitting hardcoded output values or an unchanged starter template, the resulting behavioral signals, such as fewer errors and faster completion, may appear favorable and suppress early warning alerts that instructors require. CodeInsight is designed to address this identified gap.

Synthesis of the Literature and Studies

Based on the reviewed literature and studies from relevant research works, it can be noted that remarkable advances have been made toward recognizing and solving student challenges with introductory programming courses. The studies related to failure rates reveal that there are certain underlying issues such as conceptual barriers and procedural problems at the core of poor academic performance. On the other hand, studies using learning analytics techniques show that behavioral variables like error frequencies, repeat attempts, and submission schedules can be used to identify these problems before students fail in their courses. Other fundamental concepts related to error management, post-error behaviors, AST-based code evaluation, formative assessment, and longitudinal monitoring support this approach.

However, there still exist significant gaps in all these research articles. The Philippine-based researches have proven that organizations are using digital systems, although these systems lack the ability to analyze behavioral aspects as well as the concepts for intervention. On the other hand, international research studies are much better in terms of monitoring and warning features; however, these studies are only able to examine the performance of the learners on a course/module level but not on concept level. Moreover, no system has used AST technique to verify the structure of the code written by a learner to verify the use of a certain concept.

These limitations highlight a clear research gap: There does not currently exist a system that combines behavioral analytics, detection of difficulty on the concept level, structural verification using Abstract Syntax Tree (AST) analysis, validation of hidden test cases, and real-time interventions by the instructor within a single web based framework for introductory programming classes. The purpose of creating CodeInsight is to fill this need. By collecting information about behavioral metrics including errors, retries, and time spent on tasks on a concept-by-concept basis, performing structural verification of code using AST analysis, and creating difficulty levels that generate alerts and intervention suggestions based on those difficulties, CodeInsight seeks to span this gap.

System Technical Background

The following sub-section discusses the technical background of the technology used for CodeInsight, which is a Web-Based Programming Concept Difficulty Analytics System. The current study examines the technology that can be used and the design of the software program. All the information related to the above discussion describes a comprehensive view of the technology used for realizing the purpose of the system.

Software development and coding will be carried out using Visual Studio Code – an open source programming framework developed by Microsoft, which

allows writing codes of different languages. In simple terms the software application will be written using the Visual Studio Code, which will act as the development environment for the process. It means the process of coding for the frontend and backend will be performed using Visual Studio Code integrated terminal and Git integration together with the relevant plugins for the specific languages such as JavaScript, React.js, and Node.js. In addition, GitHub will be used as a collaborative coding platform, where the whole development process from the beginning till the final stages can be managed and tracked.

For the frontend, the use of React.js will be required as the main user interface framework. The React.js library was built and is maintained by Meta and stands out due to its modular structure and optimized rendering through a virtual DOM. All pages for students and instructors in the system will be created using React.js – from the analytics dashboard, over an interface for managing exercises, to progress views of students. React.js will be initialized and bundled using Vite, which represents the new generation of frontend build tools capable of dramatically speeding up the development process due to instantaneous server startup and fast hot reloading of modules. The editor used by students inside the student coding interface will be none other than Monaco Editor. It is a browser-based code editor that has been developed by Microsoft, just like Visual Studio Code, as an individual library. Therefore, the learners shall benefit from the usage of a professional environment, featuring syntax highlighting, line numbers, and error messages inline for C++ code.

In relation to the server part of CodeInsight, Node.js would serve as a JavaScript runtime environment, employing the V8 engine used by Chrome for creating scalable applications. In regard to this, Express.js, besides Node.js, will serve as a web framework at the back end of CodeInsight, dealing with all the issues regarding RESTful API routing and communications among the front-end, database, compiler, and the analyses engine. The mentioned choices for the frameworks would fit perfectly the needs for CodeInsight since there should be managed a number of concurrent code submissions and behaviors of the learners.

PostgreSQL will be adopted as the relational database management system responsible for all persistent data storage within CodeInsight. The fact that PostgreSQL is an outstanding and free database engine that is renowned for its strength and support of full SQL language standards, it is important to note that PostgreSQL will manage all system data, including records of users, courses, their sections, students' enrollments, instructors' exercises, concept tags, code submissions, calculated values of Concept Difficulty Scores, and alerts generated.

When it comes to executing code, CodeInsight will invoke the GNU Compiler Collection, particularly G++ for compiling and running C++ code, directly on the server. Instead of outsourcing this process through a third-party sandbox execution environment, CodeInsight will run G++ compiler directly, then capture output, errors, error types, and execution results for use in calculating the concept difficulty score and analyzing the behavior using micro-concepts based rules. Direct compilation also provides the advantage of ensuring all information gathered about students' submissions in terms of behavior is precise and available. It should be noted that the choice of GCC/G++ compilers aligns perfectly with the curriculum used for C++ programming courses offered at Pampanga State University. In the case of an Abstract Syntax Tree verification feature in the CodeInsight system, tree-sitter will be employed as the parsing library to create trees that represent structural information about student submitted source code.

CHAPTER III

This chapter explains the methodologies used for designing the project system, as well as the various models. A methodical approach will follow in developing the system, which consisted of: gathering requirements, developing designs, developing systems, and implementing systems. The visuals provided include diagrams of System Structure, Data Flow, and Functionality That visually describe the architecture of the system. These diagrams will give detailed information as to how the system will work when used. In addition, they will describe how data flows through each of the various business processes within the system.

Research Design

This study will employ an explanatory sequential mixed-methods research design, allowing for the use of both quantitative and qualitative approaches to evaluate CodeInsight. In this type of research design, the quantitative phase is where the researcher collects numerical data first to analyze, and the qualitative phase further data are gathered in a second phase specifically to fully explain and elaborate on the initial numerical findings gathered in the quantitative phase. This design is deemed the most appropriate given the dual nature of the study's evaluation objective, as it allows the researcher to establish objective, measurable evidence of the system's performance while also capturing the lived experiences and perceptions of its intended users.

The objective of this study is to evaluate and find out whether CodeInsight can successfully detect concept-level programming difficulty, support instructor decisionmaking through real-time behavioral analytics, and improve student learning within introductory programming courses at Pampanga State University. The study is conducted and developed to address the need for a behavioral analytic tool that can detect concept-level difficulty in students learning introductory programming courses, and to help instructors find out which students are struggling in their learning journey by using real-time analytics. Utilizing first-year IT, CS, and IS students and their instructor in main campus of PSU, we can assess the system that

allows the researcher to produce evidence-based conclusions about CodeInsight's effectiveness and usability in an actual academic setting.

This study will utilize two phases in collecting and analyzing data. The first phase is a quantitative approach, which will evaluate whether CodeInsight successfully detects concept-level programming difficulty, supports instructor teaching method, and improves student learning outcomes by examining measurable data such as pre- and post-assessment learning gains, and system quality characteristics defined under ISO/IEC 25010, particularly Functional Suitability, Usability, and Performance Efficiency. These measurements provide evidence of the system's overall performance, thus serving as a foundation of the study's evaluation.

However, numerical results alone may not fully explain why certain patterns emerge, for instance, why a particular quality characteristic received a certain rating, or how the analytics dashboard actually influences an instructor's teaching approach. Thus, this led to using a qualitative phase to provide much deeper insight into the quantitative results. Qualitative data will be gathered through open-ended questions embedded within the survey instruments given to both instructors and students via Google Forms, with instructor questionnaires and student questionnaires designed distinctly to reflect the differing roles each group plays within the CodeInsight. The qualitative responses will then be analyzed and interpreted to figure out the pattern that emerged in the quantitative findings, allowing the researcher to move beyond the numeric result and gain a much deeper understanding of how and why users experienced the system as they did, complementing the quantitative phase to provide a complete and grounded evaluation of CodeInsight.

Software Development Model

The researcher utilizes the Agile Software Development methodology to develop CodeInsight. This methodology uses an iterative and incremental development by breaking the system functions into manageable stages, in which each stage is built upon the previous one while allowing a continuous refinement of the overall system's function.



Figure 2. Agile Software Methodology

This type of methodology is deemed to be the most appropriate for the development of CodeInsight as it is a feature rich web system, such as the web-based code editor, Concept Difficulty Score engine, AST-based code structure verification using tree-sitter, analytics dashboard, and academic integrity monitoring layer, each of which is developed in each successful stage, each producing a functional increment of the overall system. Rather than delivering the entire system at once, each stage produces a functional increment, ensuring that issues are identified and resolved early and controlled in a systematic manner.

1. Requirements

This stage defines the specific functional and technical needs of CodeInsight in its development. In this stage, the researcher has identified and outlined the functional requirements for each major system component, such as the concept difficulty formula. This stage also specifies the behavioral tracking metrics of CodeInsight, especially the student error frequency, the retry count, and the time-on-task, as well as the CDS computation rules, concept tagging mechanism, academic integrity requirement, and analytics dashboard features. CodeInsight is limited only to introductory C++ programming courses for the Academic Year

2025-2026. This will only target the seven C++ programming concepts, which are the: datatypes, variables, conditionals, loops, Functions, arrays, and object-oriented programming. The primary code execution engine of CodeInsight will be established as GCC C++ . The Sequential Code Processing and the analysis pipeline of CodeInsight will be established as one of the system requirements. This will establish the flow from the GCC compiler, through the AST analysis using tree-sitter, into micro-concept rule analysis, in this will be followed by the academic integrity checking, and lastly the CDS computations.

2. Design

Concrete system blueprints will be made through this stage based on the established specification. This includes the design of the web-based code editor interface, the exercise management workflow for instructors, including the concept tagging mechanism that links each exercise to one or more of the seven programming concepts, and the analytics dashboard layout, featuring the difficulty heatmap, early warning alerts, individual student profiles, longitudinal progress reports, deterministic micro-concept difficulty analysis, and academic integrity. To ensure that the system's logic is fully defined before development begins, the design stage further formalized the distinction between visible and hidden test cases and the Run and Submit operation modes. The Run mode will be designed to test student code against visible test cases only, without recording behavioral metrics, while the Submit mode will be designed to run code against all test cases, including hidden ones, while at the same time collecting behavioral data for CDS computation. The AST-based code structure verification pipeline using Tree-Sitter and canonicalized pattern matching from IBM's Project CodeNet, filtered to approximately 15,000 introductory C++ solutions, will also be formalized during this stage, along with the safety rules governing edge cases in the CDS computation. The GCC C++ compiler will also be incorporated into the system design as the execution engine, with its compiler error patterns serving as one of the primary signals for micro-concept difficulty analysis. This ensures that the system's logic and structure are fully specified before development begins.

3. Development

After the blueprint has been finalized in the system design Stage, the third stage is then started, where the development process is carried out across multiple development stages, where each stage focuses on implementing a one specific function of the system. The first development stage of CodeInsight has built one of the core foundation infrastructure of the CodeInsight system, which will include a role-based authentication and the initial database configuration that supports behavioral tracking and concept tagging. The second development stage of CodeInsight will build the web based code editor and the exercise management system, this includes the run and submit method and the instructor exercise maker with concept tagging for the seven C++ programming concepts. The third development stage of CodeInsight will focus on the development of the CDS engine, that implements the behavioral formula of $CDS = (0.40 \times NER) + (0.35 \times NRS) + (0.25 \times NTS)$, alongside the development of an AST-based code structure verification pipeline that uses Tree-Sitter to validate the submitted C++ code against canonical patterns that are found and derived from IBM's project CodeNet, which are filtered to approximately 15,000 introductory C++ solution. Integration of the CodeInsight's GCC C++ Compiler along with its error patterns will also take place during the third stage as the main code executor, whose information will be provided to the micro-concept difficulty analysis engine, together with other AST structures and test failure patterns. The fourth development stage of CodeInsight will build the analytics dashboard and its visualizations, including the difficulty heatmap, the early warning alerts, student individual profiles, the longitude progress report, the deterministic micro concept difficulty analysis, and the structure violation report panel. The fifth and final development stage of CodeInsight will incorporate the academic integrity layer, behavioral, and code growth anomaly detection, and a student interface for performance profiling, which allows students to view their own concept difficulty profiles and track their progress throughout the course, and passive behavioral logging. Resulting in each stage producing a functional and testable system increment

4. Testing

The testing stage covers the comprehensive testing of each system function and the integrated system as a whole. Unit testing validates individual functions of the system, such as the GCC C++ compiler integration, CDS computation engine, and the AST verification pipeline using Tree-Sitter. The complete Code Processing and Analysis Pipeline will also be verified during this stage to ensure each stage operates correctly and in proper sequential order. Integration testing will ensure that behavioral data will show up correctly from student code submission through to the analytic dashboard. System functional testing will be conducted to confirm that all system features are aligned with the requirements that are defined in stage 1, including the correct functioning of the seven concept tags, the two submission modes, the academic integrity monitoring, and the analytics dashboard visualizations. The system will also be assessed with respect to ISO/IEC 25010 quality characteristics, namely Functional Suitability, Usability, and Performance Efficiency, through the System Usability Scale (SUS) assessment tool by alpha testers. All problems that may be encountered within this stage will be addressed in order for the system to move on to its Deployment stage, thus ensuring stability and reliability.

5. Deployment

The fifth stage is where CodeInsight is ready for deployment for its intended use within the introductory C++ programming course at Pampanga State University. The CodeInsight will be deployed on using Oracle Cloud's always-free ARM virtual machine (VM.Standard.A1.Flex: 4 cores, 24 GB RAM, 100 GB disk), which is running Ubuntu 22.04, with Node.js, PostgreSQL, and G++ as the GCC C++ compiler runtime for server-side code execution, Nginx, and PM2 configured as the core server stack. A domain name will be provided by DuckDNS for free, and for security, HTTPS is implemented using Let's Encrypt. This deployment technique will make CodeInsight to be fully accessible to instructors and students at any location with an internet connection, with no ongoing cost, and without requiring the

institution's local server. This will allow users to have consistent access to a reliable and functional system.

6. Review

In this Review stage, the researcher will conduct feedback for CodeInsight's users. The questionnaire will assess the system's functional suitability, usability, and performance efficiency using a structured questionnaire based on the ISO/IEC 25010 quality model for the first-year students and faculty of the Information Technology (IT), Computer Science (CS), and Information Systems (IS) programs of Pampanga State University, Main Campus. In order to ensure that the assessment results are based on actual experience with the installed system and not just assumptions about its capabilities, the System Usability Scale (SUS) test, pre-test, and post-test assessment surveys will be conducted among instructors and students in this stage. The student will evaluate the CodeInsight using the information obtained from their personal experience of working with the web-based code editor, concept difficulty profiling, and progress tracking features. On the other hand, instructors will rate the software according to their experience working with the analytical dashboard, early warning system, micro-concept difficulty analysis, and academic integrity monitoring. Additionally, open-ended responses were collected using Google Forms to give users the opportunity to provide qualitative feedback about their experience with CodeInsight. The results of this process will assist in the determination of the system's strengths, weaknesses, and areas for improvement.

Requirements Analysis and Documentation

The present chapter describes the analysis and documentation of the required system, referred to as CodeInsight: A Behavioral Analytics-Driven Web System for Concept-Level Difficulty Detection and Early Intervention in Introductory Programming Courses. This chapter is a description of both functional and nonfunctional requirements, which will form the basis for design, implementation, and evaluation of the system. The introductory courses in programming still have high dropout rates because of students' inability to comprehend programming

fundamentals. One of the main reasons behind this challenge is the absence of technology that can detect problems that learners are having at a very micro-level. Most tools available today concentrate only on results but not on the whole process of learning.

Moreover, even though analytics has been proven to be capable of analyzing student behavior, the existing systems fail to link these behavioral patterns to specific programming concepts. Consequently, there appears to be a need for an integrated solution that will not only be able to measure behaviors but also correlate these metrics to programming concept difficulty levels. CodeInsight, the proposed web-application, offers a solution by leveraging behavioral analysis, program structure verification, and deterministic rule analysis to offer insight into student progress in real time. This chapter describes the requirements of CodeInsight in order to fulfill this goal.

Requirements Specification

Functional Requirement

Functional requirements refer to the actual tasks that the system must be able to accomplish. The requirements will come from the objectives of the research project as well as from the features of the system.

The system will have an authentication feature which will enable the users to register and log into the system based on the assigned roles. The role-based access control will be implemented in such a way that only the users assigned to specific roles will access relevant functionalities.

The system shall facilitate the ability of educators to generate and administer programming tasks. Programming tasks shall be associated with at least one concept that includes variables, conditionals, loops, functions, arrays, and object-oriented programming. The generation of visible and invisible test cases will also be included in the system to assess students' code submissions.

The system shall provide a web based editor for students to compose, run, and submit codes. Two methods will be used to run the program: the first being the Run method, which runs the program against visible test cases without collecting any behavioral metrics; the second being the Submit method, which runs the program against all test cases and collects behavioral metrics.

This should involve having the system monitor instances where the solution was copied from some source and not coded by the student. It would also include having the system identify cases where the student copied templates in their responses and where there were irregular patterns in the answers. Lastly, the system should provide students with the ability to analyze how they have been performing in terms of their mastery of concepts.

The system will generate a Concept Difficulty Score (CDS) using the collected behavioral metrics. The CDS will provide an indicator of how difficult it is for the student to learn a particular programming concept. The CDS will be recalculated every time a student submits something correctly. An analytics dashboard will be made available to educators for analyzing their students' performances. These will include difficulty maps, student performance profiles, warning flags, and progress trends. The analytics dashboard will also give information about micro-concepts that have been inferred using rule-based systems for analyzing students' behaviors and codes.

The system shall obtain behavioral metrics when students interact with the code editor. These will include measures of error frequency, number of attempts, and task completion time. The metrics thus obtained will be stored along with the relevant programming constructs for analysis purposes.

To ensure that the metrics are valid, the system shall check the structure of the code through the use of AST analysis. This will confirm the use of necessary programming constructs within the student's submissions.

Non-Functional Requirements

Non-functional requirements refer to the characteristics and performance capabilities that should be present in the system. The system will demonstrate performance efficiency through code submission and analytics generation in real-time. It will be able to support several users simultaneously with no considerable impact on performance.

Usability is very important, hence the need to design a user-friendly interface to facilitate interaction between users, which includes both instructors and students. Navigation will be seamless and feedback provided will be helpful. In addition to ensuring accuracy in calculations of Concept Difficulty Scores, among others, there will be adequate mechanisms put in place to avoid any error that could cause failure.

The security requirement will be met by securing data using encryption techniques and implementing role-based access control. The system shall ensure protection of critical data and unauthorized access. The maintainability requirement will be satisfied by adopting a modular architecture design that facilitates upgrades and modifications of the system. This ensures that the system can accommodate future developments.

The portability requirement will be accomplished by developing the system as a web-based application that can be accessed using any browser. This ensures that the system can be accessed from anywhere. These requirements match those stated in the ISO/IEC 25010:2011 software quality criteria used for evaluating the system.

User Requirements

User requirements represent the needs and wants of the system's end users in an easy-to-understand language. The user requirements are based on the problems found in the introduction to programming courses and are centered around the role of the system in fulfilling the goals of both the students and the teachers.

The system should give the students a chance to program, test, and submit their code easily. The students need real-time feedback regarding their code as well as information about their performance in terms of programming concepts. It should be possible for the system to let the students know about their performance trends over time and help them recognize the programming concepts they find hard. Furthermore, the system should have a friendly and easy-to-use interface for the students.

From the perspective of an instructor, the following functionalities are essential in the system: Creation of programming assignments and management thereof. Detailed analytics of student progress on a per-student basis as well as a perclass basis. Alerts on students who need help early enough to allow intervention. Guarantee that work submitted by the students is their own and not plagiarized.

The administrator needs to have a platform that provides for account administration, system monitoring, and security of data. The administrator requires an environment that guarantees accessibility, reliability, and efficiency. In general, the system should be accessible, reliable, and efficient to ensure that all users are able to accomplish their objectives without any hassle.

System Documentation

As presented in the system documentation of the study “CodeInsight: A Behavioral Analytics-Driven Web System for Concept-Level Difficulty Detection and Early Intervention in Introductory Programming Courses”, there is a series of systematic diagrams that can be used to create an illustration of the intended system. This series of diagrams will serve as the tools for analysis and design of the system. The diagrams provide a visual presentation of the structure of the system and how it works.

Data Flow Diagram (DFD)

The following is a list of some important diagrams included in this section, which include the Context Diagram, Data Flow Diagram (DFD), Use Case Diagram, Entity-Relationship Diagram (ERD), System Flowchart, and System Architecture Diagram. The above diagrams indicate the different components of the system, which include data flow, use case, database relationships, flowcharts, and system architecture, respectively. These diagrams, together with their descriptions, make the design of the system more precise and accurate.

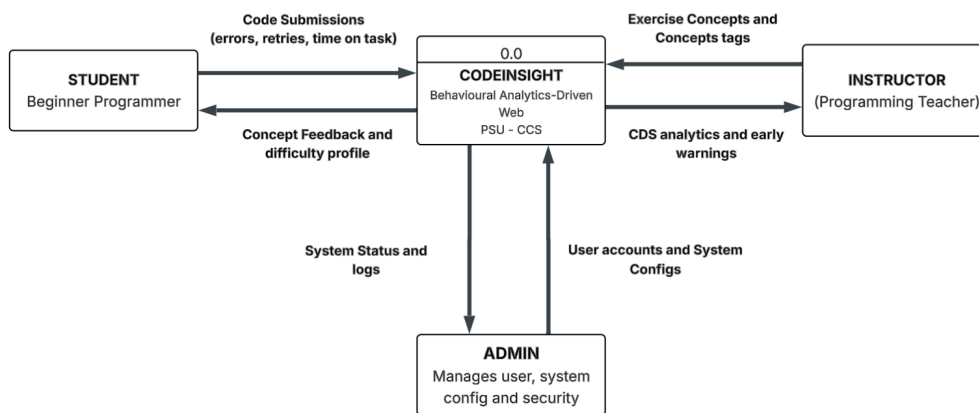


Figure 3. Context Diagram

The Context Diagram shows the whole system as one big process named CodeInsight Behavioral Analytics-Driven Programming Learning System.

The first external entity in this system is the Student (Beginner Programmer). The interaction of this external entity with the system happens through submission of the code for evaluation. The student will give inputs to the system like the program code written, which may contain errors, reattempt codes, and correct codes as well. In exchange, the system will provide outputs like concept-based feedback and difficulty profiling.

Secondly, there is the Instructor (Programming Teacher) who engages with the system through inputting the exercise concepts and tagging the programming topics. The instructor offers inputs in the form of learning content and exercises that are categorized according to certain concepts, which inform the analysis of the

system. Some of the outputs include Code Difficulty Scoring (CDS) analytics and alerts, which enable the instructor to track the performance of learners.

Finally, there is the Administrator whose role involves controlling the entire operations of the system. He provides inputs in terms of users' accounts details, system settings, security, among other relevant information that helps in the functioning of the system. Outputs from the system may involve monitoring of the state of the system.

Arrows show the path through which the data travels between the system and the external entities. The data moves from the student to the system in the shape of submitted codes, from the instructor in terms of exercise ideas and tagging, and from the administrator in relation to system settings. On the other hand, the data flows from the system to the student in the form of feedback and difficulty levels, to the instructor in terms of analysis and early alerts, and to the administrator in terms of system states and log reports.

Data Flow Diagram (DFD) Level 0

Explanation for the Level 0 Data Flow Diagram for CodeInsight, shows all the procedures involved in the working of the system. The system is broken down into sub-tasks, which include user identification, inputting of code, code analysis, giving feedback, and managing the system. The process of interaction and flow of information from external entities (Student, Instructor, Admin), system process, and database can all be explained using this diagram.

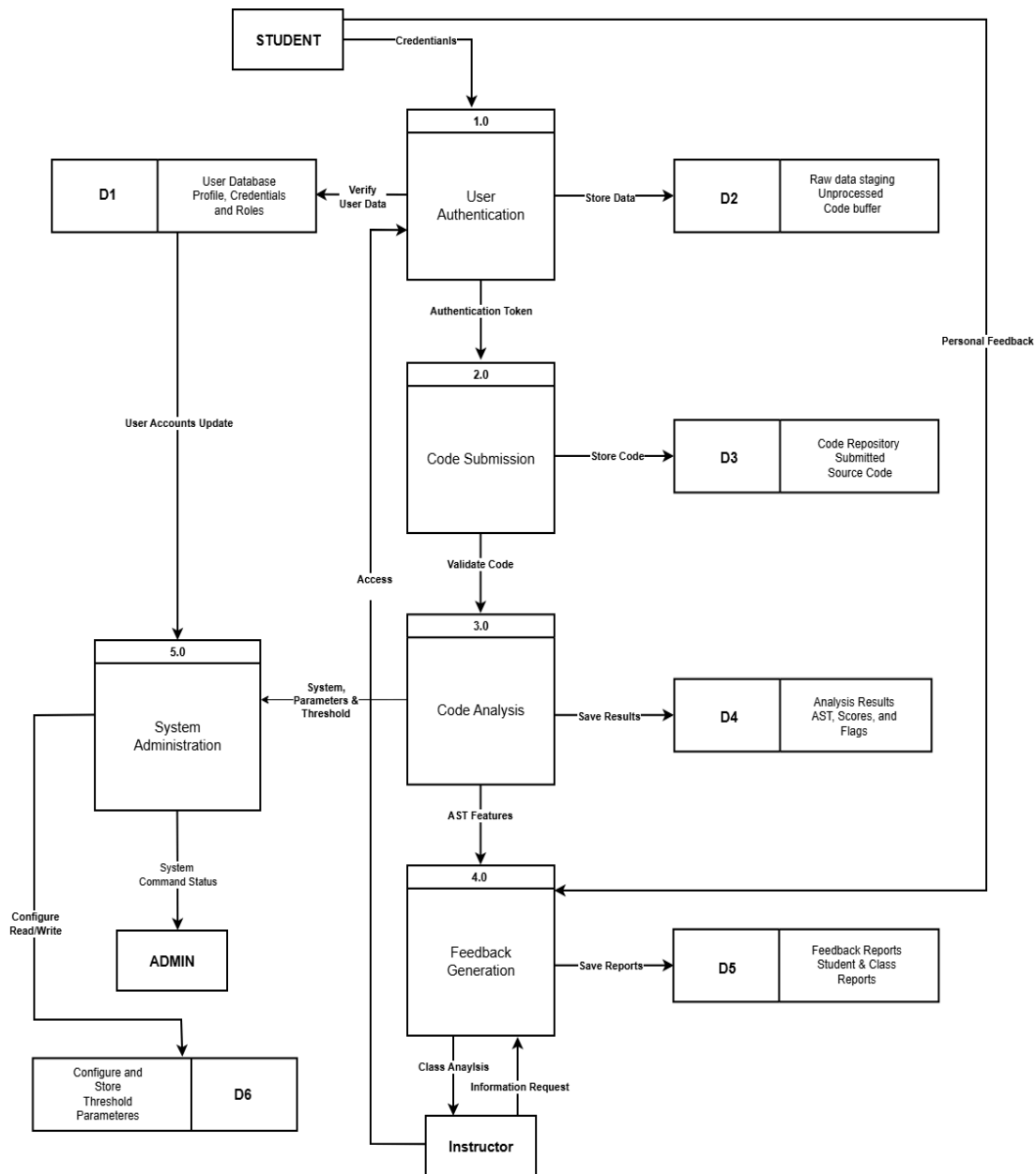


Figure 4. Data Flow Diagram(Level 0)

The level 0 data flow diagram of the CodeInsight system shows the flow of information through the system. It describes the processes carried out by the system including user identification, code submission, code analysis, feedback generation, and system administration. It also describes the flow of data from the user to the process and data store. The level 0 DFD is used to demonstrate the relationship

between the system and the external entity.

Process 1.0 – User Authentication Process. This process entails the authentication of the user. The students, tutors, and administrative staff authenticate into the system. This process entails the authentication of the user from the DS1 User Database that consists of the profile, credentials information, and role of the person. After successful authentication, the authentication token is generated.

Process 2.0 – Source Code Upload. During this step, the code from the Student will be uploaded into the system. The process does input validation, checks for completeness and prepares the code for analysis. All the codes that are uploaded during this process will be held at the temporary storage area DS2 Code Repository. After the validation process, the codes will be forwarded to the analysis process.

Process 3.0 - Automated Analysis Engine is tasked with analyzing the source code with the help of different automated methods. Here, the system will conduct an Abstract Syntax Tree (AST) analysis, as well as a syntactical and structural analysis of the code, in order to determine its correctness and structure. Apart from checking the correctness of the software, the test also examines its quality and complexity. This will lead to the production of structured data which will then be stored in DS4 Analysis Result. The structured data will consist of the extracted features, the score card, and the integrity flags.

Process 4.0: Feedback generation is very essential for transforming the information acquired during the analysis stage into insightful and useful feedback. During this process, the system produces different types of data, such as student feedback reports, instructor analytics, graphs, and integrity indicators. This helps to create individualized feedback and collective data for the benefit of the students and instructors respectively. This feedback data is then saved at the DS5 Feedback Report database. Personalized feedback is sent to individual students while instructors are free to request class analytics to gauge their student's performance.

Process 5.0 – System Administration involves the overall management and maintenance of the system by the Admin. In this regard, the administrator plays the role of controlling user accounts, setting up system parameters, threshold values, and maintaining the database. This will guarantee the efficient operation of the system and ensure that its purpose is achieved. The configuration information is stored in the DS6 Config Store, and any modification done by the administrator will affect the behavior of the system.

Entity Relationship Diagram (ERD)

In the given ERD (Entity Relationship Diagram), the logical structure of the database of the proposed CodeInsight system is presented. The ERD provides a visualization of the entities along with their attributes, as well as shows the connections between these entities. The ERD plays a key role during the process of requirement analysis as it acts as a basis for efficient data storage and management in the system. The ERD reflects all the main functions of the proposed system including code submission, code analysis, assessment of its performance, and generation of feedback.

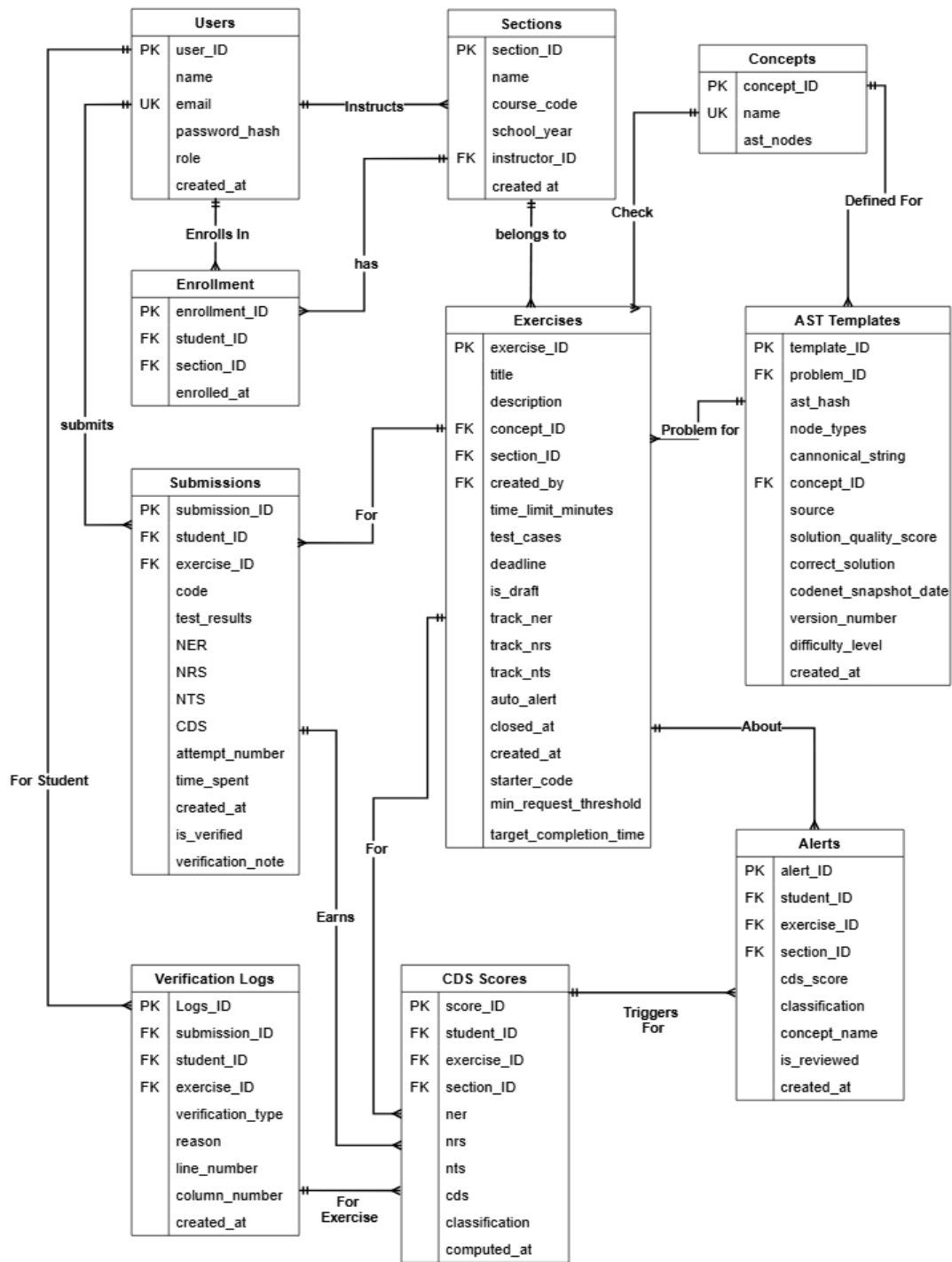


Figure 5. Entity Relationship Diagram

The ERD displays the structure of data in the CodeInsight application through the identification of key entities, which include users, sections, exercises, submissions, and analytical tools. The entities are linked together in order to achieve functionality in the application, especially with regards to submissions, performance evaluation, and insight generation.

The USERS table contains information about the users of the system, which can be either students or faculty members. The columns of the USERS table include the following: user_id, name, email, password_hash, role, and created_at.

The SECTIONS entity is used to secure the grouping structure that defines where exercise assignments are made. The following are some of the attributes that will be assigned in this entity: section_ID, name, course_code, school year, and created_at as well as the instructor_ID.

ENROLLMENTS entity captures how students relate to different sections. It will use the attributes such as enrollment_ID, student_ID, section_ID, and enrolled_at.

The EXERCISES entity represents programming tasks assigned to students. It includes attributes such as exercise_ID, title, description, concept_ID, section_ID, created_by, time_limit_minutes, test_cases, deadline, is_draft, track_ner, track_nrs, track_nts, auto_alert, closed_at, created_at, starter_code, min_request_threshold and target_completion_time . Exercises define the problems that students must solve and contain evaluation parameters.

The CONCEPTS entity stores programming topics associated with exercises. It includes attributes such as concept_ID, name, and AST_nodes. This helps categorize exercises based on learning objectives..

The AST_TEMPLATES entity stores predefined structures used for code analysis. Attributes included are template_ID, problem_ID, ast_hash, node_types, canonical_string, concept_ID, source, difficulty_level, solution_quality_score, codenet_snapshot_date, version_number, correct_solution and created_at. This entity supports automated analysis by comparing submitted code structures.

The SUBMISSIONS entity contains student code submissions. It includes the following attributes such as submission_ID, student_ID, exercise_ID, code, test_results, passed, NER, NRS, NTS, CDS, attempt_number, time_spent, created_at, is_verified and verification_note. This entity is central to the system as it captures all student activity.

The VERIFICATION_LOGS table captures all the validation information along with the review information about any submission. It consists of log_ID, submission_ID, student_ID, exercise_ID, verification_type, reason, line_number, column_number, and created_at fields.

The CDS_SCORES entity stores computed performance metrics. It includes the attribute score_ID, student_ID, exercise_ID, section_ID, NER, NRS, NTS, CDS classification and created_at. This replaces traditional grading by providing multidimensional evaluation.

The ALERTS entity stores notifications generated based on performance. It includes the attributes alert_ID, student_ID, exercise_ID, section_ID, cds_score, classification, concept_name, is_reviewed and created_at. Alerts help instructors monitor students who may need attention.

In summary, the ERD provides a solid and flexible architecture for the database design that ensures efficient processing of user data and analysis of students' performance, as well as automated feedback on programming problems.

The ERD is crucial for the proper functioning of the system and its efficiency as an intelligent learning environment for programming courses.

Use Case Diagram

The Use Case Diagram illustrates the interaction between two primary actors: the Student and the Instructor, with the Code Structure Score at the center and a set of automatic use cases that are triggered by the system itself. The Student interacts with the system mainly for learning and self-monitoring purposes. After logging in, students primarily use the system for the purpose of learning and monitoring their own performance. Once logged in, the student is able to access the available exercises, compose and execute code through the system's built-in editor, and submit their answer whenever they are ready. At the moment a submission is made, the system on its own automatically carries out two processes, which are the assessment of the code structure and the generation of the Code Structure Score or CDS. In addition to completing their work, students can access their CDS profile to examine previous entries and track their own progress over time. Students may monitor how they are progressing through the course and identify particular areas where they need to improve.

CODEINSIGHT

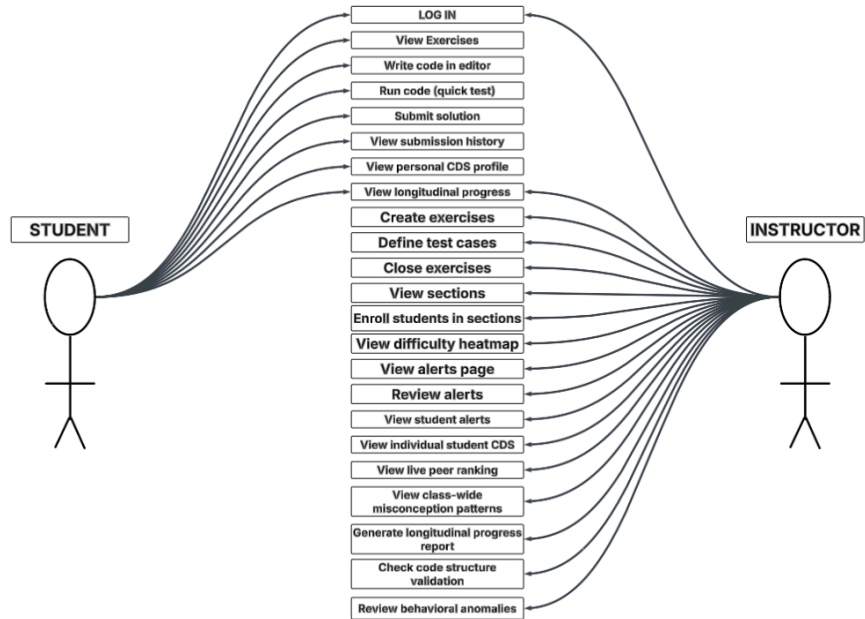


Figure 6. Use case Diagram

The instructor uses the system primarily for management and monitoring. They can log in, make exercises, set up the test cases that will be used to grade student submissions, and close exercises when the activity period is over. When an exercise is closed, two built-in use cases are automatically triggered: batch CDS computation and alert generation. Aside from these, the instructor can also view sections, enroll students, check the difficulty heatmap, review alerts, view individual student CDS scores, see live peer rankings, identify class-wide misconception patterns, generate longitudinal progress reports, validate code structure, and review behavioral anomalies.

System Flowchart

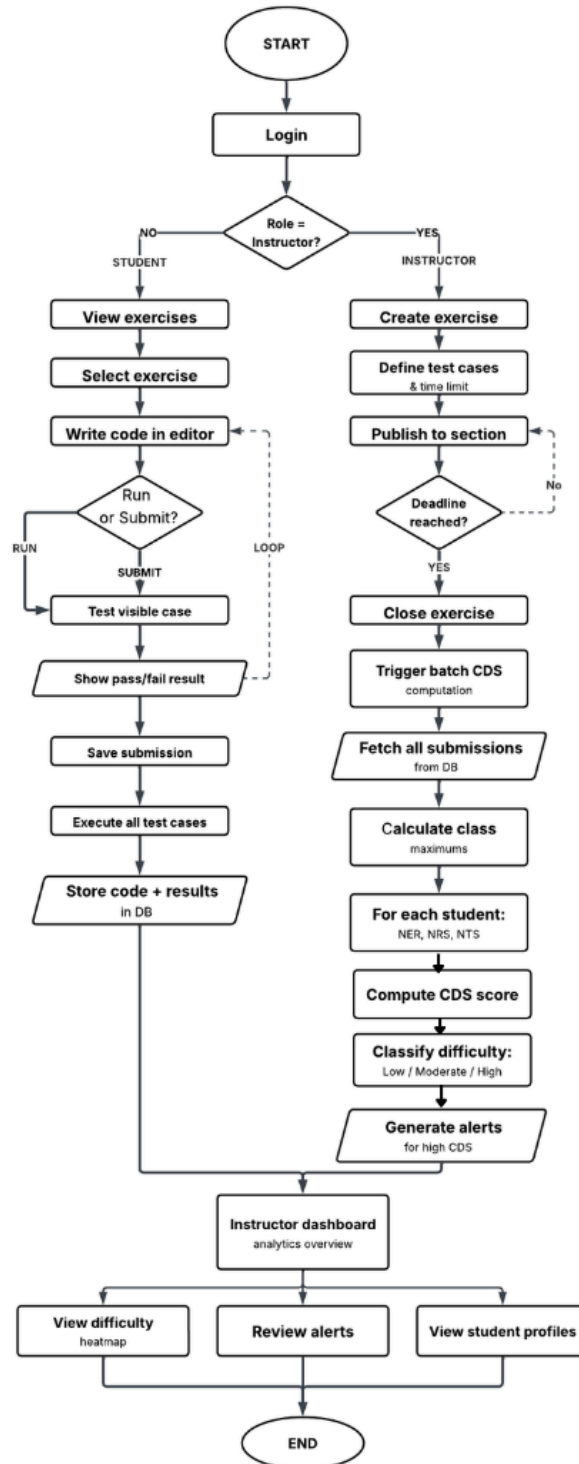


Figure 7. System Flowchart

This flowchart shows the operational flow of an educational coding platform where users are presented with two different flows depending on whether they are logged into the system as a student or an instructor. After logging into the system, the algorithm checks the user's account type and finds the appropriate flow to follow. For instructors, the flow follows these steps to create exercises, define test cases (with time limits), and publish the created exercise to specific sections. After a certain time period, when an exercise has been closed, the instructor's workflow continues in a data-processing phase, referred to as "Trigger batch CDS." This process includes retrieving all student submissions for the completed exercise, determining the maximum class metrics for the completed exercise, calculating the individual metrics for each student (NER, NRS, NTS), and determining how difficult the completed exercise is (utilizing a CDS) to send the instructor alerts about any students who have not performed well on a particular task. Once the data for the created exercise has been accumulated, instructors can view the compiled data through an Instructor Dashboard, which includes difficulty heatmaps, a list of alerts, and the associated student profiles.

Students can see what exercises are available to pick from on one side of the workflow then enter code within one integrated code editor. After the code has been entered, they can run the code which will only test the first visible test case and will display any feedback, or they may submit the code for evaluation. Once submitted, the system will save the code, execute all of the backend test cases, and save both the submitted code and the results to the code repository. This code and results will, over time, be used to generate batch reports by the instructor, which will provide the final dashboard metrics regarding student performance in relation to analytics and difficulty levels prior to the end of each batch process.

System Architecture Diagram

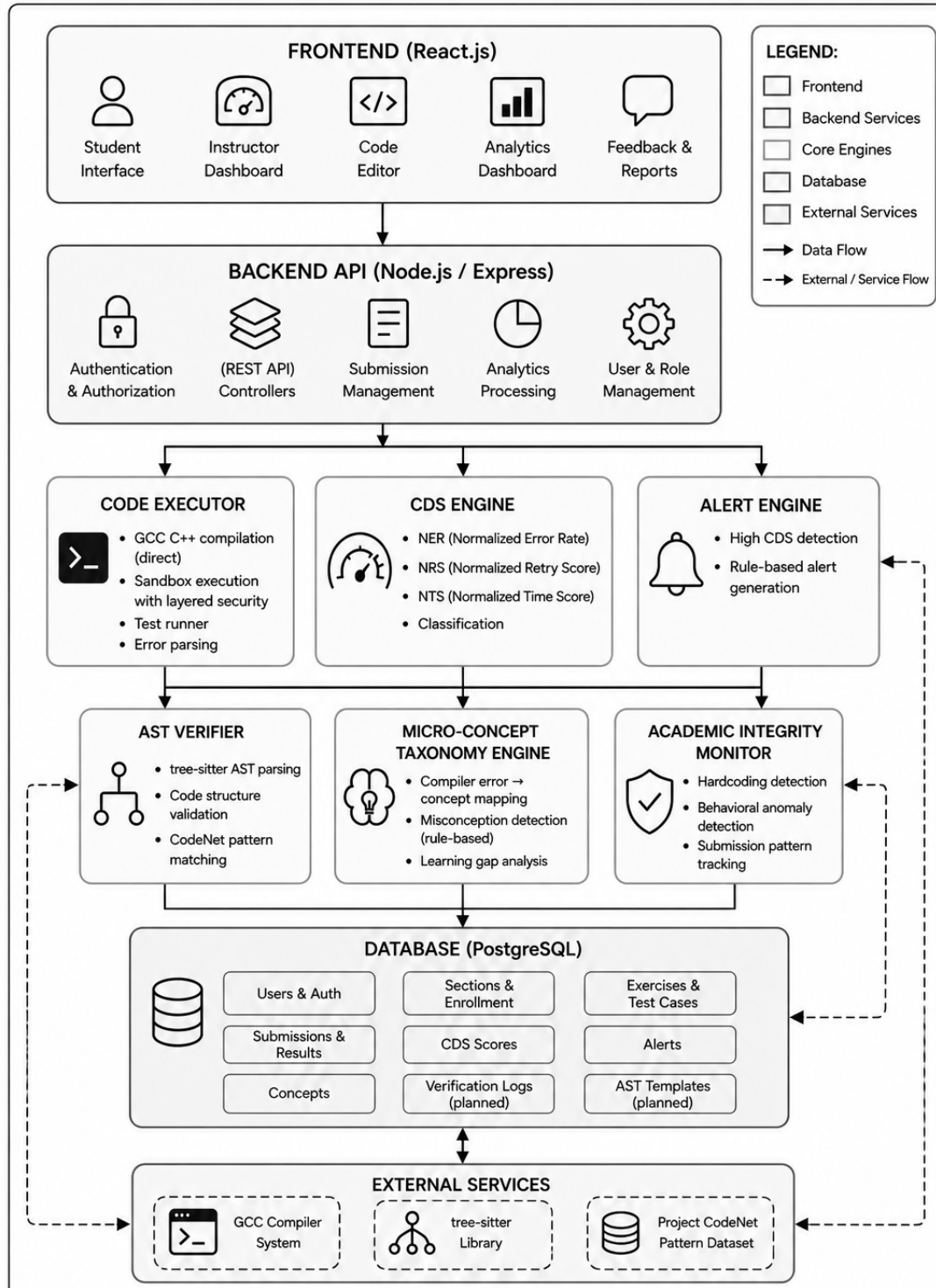


Figure 8. System Architecture

The diagram shows an architectural view of the CodeInsight system; which is an automated code analysis and feedback system for education of programming. The different components of the system include frontend, backend API, core engines, database, and external services along with the data flow between them.

The frontend layer consists of React.js that serves as the UI layer for both student and instructor. Student Interface, Instructor Dashboard, Code Editor, Analytics dashboard, feedback and reporting are modules under the frontend layer which facilitates to write, submit and view the code as well as monitor their progress and receive feedback from the system.

The frontend layer communicates with the backend API (built in Node.js and Express), which serves as the central control unit of the system, authenticating and authorizing users, handling the REST API requests from the frontend, facilitating the submission of code and processing, handling the analysis and computation of data and providing the management for user and roles.

There are various engines at the core of the system for running and analyzing the code. Code Executor runs the submitted code in a sandbox environment after compilation using GCC C++ and supports execution of test cases and error parsing. CDS Engine(Code Diagnostic Score Engine) uses the results from the test cases to compute metrics such as Normalized Error Rate (NER), Normalized Retry Score (NRS) and Normalized Time Score (NTS), and classifies the students on the basis of their coding performance. The Alert Engine identifies high-risk coding performance patterns and triggers alerts.

Behind the scenes, are the deeper analysis components: AST Verifier checks the code structure for violations using a tree-based analysis and code structure parsing, which allows identification of code patterns. The Micro-Concept Taxonomy Engine maps compiler error outputs to individual programming concepts

and detects conceptual misunderstandings based on rules. The Academic Integrity Monitor works on detecting hardcoding and behavior anomalies of submissions.

The PostgreSQL database is used for storing all data related to users, their submissions and scores, assignments and tests, system configurations etc. It consists of sections and enrollments, and submission details, CDS scores, exercises and test cases etc. Additional storage required for verification log, AST template will be used for deeper analysis.

External services that integrate with the system include the GCC compilation system, the tree-sitter library and the Project CodeNet for benchmarking and analysis.

The system also integrates external services, including a GCC compiler system for code execution, a tree-sitter library for syntax parsing, and the Project CodeNet dataset for pattern matching and benchmarking. These integrations enhance the analytical capabilities and accuracy of the platform.

Research Tools and Instrument

The study used different ways to do research and find out things to help make CodeInsight work. CodeInsight is a system that looks at how people behave. The people in charge picked tools to help students at Pampanga State University who are learning to program. They want to help these students find problems they are having and get help early. They have to be careful when they pick these tools. The tools need to be able to see how students are coding and give them help when they need it. This part of the study includes making the software collecting data and figuring out how to assess things. They want to make sure each part of CodeInsight works correctly so it can do what it is supposed to do.

The way CodeInsight monitors user progress is based on combining data with analysis datasets. This technology helps find programming concepts that students

struggle with by looking at how they code in real-time and their performance stats. By adding these tools the system gets a way to support students using data making the research results more reliable and trustworthy in an educational environment. CodeInsight uses this data to understand user progress and provide support. The system relies on data to help students. This makes the study findings more valid. The combination of monitoring and analysis datasets is key, to how CodeInsight works

1. Software Tools

The study needs specific software tools which are essential to create and operate and control the proposed system. The tools function like an engine which enables the system to handle data processing and data storage and data display functions. The system enables teachers to monitor student progress while students receive immediate assistance. The system will be developed through the use of web development technologies. The frontend layer consists of React.js that serves as the UI layer for both student and instructor. This approach enables the frontend to adapt automatically for different types of devices. The backend system will operate through the utilization of Node.js and Express programming languages. The system will utilize these tools to analyze user behavior patterns while storing information. The objective of the system development process is to create a user-friendly system which delivers seamless functionality together with exceptional user experiences. The system enables teachers to monitor student development while delivering instant support to students. The software tools are essential for creating a system that works well. The system will use these tools to process and show data. The system will provide teachers and students with all necessary information.

The integration of particular libraries and APIs will enable the development of Behavioral Analytics and Concept-Level Difficulty Detection. The tools will analyze patterns which include code compilation errors and time spent on tasks and logical progression of work to find particular points where students experience difficulties. The system provides teachers with user-friendly dashboards which

utilize data visualization packages like D3.js and Chart.js to display information. The system uses these tools to monitor student activities and detect learning plateaus while providing immediate intervention alerts which help students succeed in their introductory programming courses.

2. Data Collection Instruments

This study will use data gathering instruments for students and teachers to see if the system fits the educational needs and real classroom situations in Pampanga State University.

3. Observation

Observation is used to investigate the students' use of integrated development environment (IDE) and web-based coding platforms in the computer laboratories of Pampanga State University. This method allows researchers to directly examine coding activity, such as common syntax errors, time spent on certain logic blocks, and the frequency of compilation attempts. Observation is a nice way to corroborate the data collected from system logs and offer context to the difficulties of introductory programming. This tool helps to construct a more responsive and concept-aware early intervention system by recognizing certain behavioral patterns and inefficiencies in students' manual debugging procedures.

4. Evaluation Instrument

The study employs assessment instruments such as questionnaires and system testing to ensure the efficacy, usability and general quality of the established system. The instruments provide subjective and objective measures of system performance.

4.1 Questionnaire

The produced system is evaluated using the software quality model ISO/IEC 25010 with a structured questionnaire. The questionnaire is developed in a 4-point Likert scale that allows respondents to score the system according to certain quality aspects like functional appropriateness, usability, and dependability and

performance efficiency. The instrument is given to the selected students as well as the IT faculty members who have used the CodeInsight platform. They can submit feedback based on their actual experience with the difficulty detection features and the intervention prompts. The questionnaire data is examined to identify the strengths and shortcomings of the system to make sure that it fulfills the quality criteria set by the College of Computing Studies and pedagogical expectations of basic programming courses.

4.2 Testing

System testing is carried out to assess the functionality, performance and dependability of the developed system in actual classroom operational settings. Functional testing is an important step to verify that all components of the system work correctly, e.g. behavioral tracking, concept level analysis. We also do performance testing to measure the responsiveness of the system to real-time coding telemetry and the accuracy of the early intervention signals. Furthermore, user acceptability testing (UAT) is performed on many web browsers and devices to simulate realistic student usage scenarios and ensure consistent and correct platform operation. The test findings are objective evidence of the reliability of the system and assist the validation of the effectiveness of CodeInsight as an instructional tool driven by behavioral analytics.

Statistical Treatment of Data

The use of suitable statistics will be done in order to properly analyze and interpret the data gathered from the respondents regarding the development of CodeInsight: A Behavioral Analytics-based Web System for Concept-Level Difficulty Detection and Early Intervention in Introductory Programming Courses at Pampanga State University. The data will come mainly from a questionnaire distributed among selected respondents, specifically the alpha testers composed of CCS professors and IT experts, and the beta testers composed of freshmen students who take introductory courses on C++.

The implementation of statistical methods will assist in analyzing the quality features of CodeInsight according to the software quality standards as indicated by the ISO/IEC 25010:2011 model, which include Functional Suitability, Usability, and Performance Efficiency. The implementation of statistical methods will offer a sound scientific approach to the interpretation of users' feedback on the performance and acceptability of the developed system. This will increase the validity of research results.

Type of Data Collected

The experiment predominantly makes use of quantitative data gathered from the respondents via an effective questionnaire designed based on a 4-point Likert Scale. The two categories of respondents involved in this experiment include freshman C++ programming students used as beta testers and CCS professors along with IT experts functioning as alpha testers. These respondents rate the functionality of CodeInsight by using it and evaluating different aspects of this product including the editor, Concept Difficulty Score (CDS) engine, analytics dashboard, academic integrity layer, and progression functionalities.

The data collected in this study include numeric scores representing the extent to which the participants agree or disagree with the given statements about the features of the system. The collected data will be used to evaluate different characteristics of the CodeInsight system, including its functionality as an indicator of programmatic difficulty at the conceptual level, user friendliness, and performance efficiency.

4-Point Likert Scale Interpretation

The research uses a 4-point Likert scale to determine the degree of agreement with regard to the performance and quality of the CodeInsight system that will be created. The use of this scale excludes any possibility of choosing a neutral

response, which compels respondents to give a strong evaluation of the system. The 4-point Likert scale will be the basis for interpreting responses to the questions asked. The 4-point Likert scale is as follows:

Scale	Numerical Value	Interpretation
4	3.26 – 4.00	Strongly Agree
3	2.51 – 3.25	Agree
2	1.76 – 2.50	Disagree
1	1.00 – 1.75	Strongly Disagree

This scale is used to interpret the computed weighted mean values, allowing the researchers to determine the level of acceptability and quality performance of the CodeInsight system based on respondents' evaluations.

Statistical Tools and Techniques

To analyze the collected data, the study employs several statistical tools and techniques that facilitate accurate computation and interpretation of respondents' evaluations.

4.1 Frequency and Percentage

Respondent frequency and percentages are utilized to characterize the distribution of respondents and the summary of their responses to each survey question. The use of these tools facilitates an easy understanding of the response's frequency and its share in the total number of respondents. The use of this technique can be highly beneficial in describing the profiles of alpha and beta testers as well as analyzing the common trends in their responses to the quality features of CodeInsight. The percentage is computed using the formula:

$$\text{Percentage} = (f / N) \times 100$$

Where:

f = frequency of responses for each scale option

N = total number of respondents

4.2 Weighted Mean

A weighted mean has been adopted as the main statistical method for calculating the average score by the respondents on each of the evaluation criteria under the ISO/IEC 25010:2011 quality characteristics. The measure yields a numeric figure that reflects the perception of the respondents concerning some aspects of the CodeInsight product, including its Concept Difficulty Score engine, the dashboard functionality, and the real-time behavioral tracking ability.

The weighted mean is computed using the formula:

$$\text{Weighted Mean (WM)} = \Sigma(f \times x) / N$$

Where:

f = frequency of responses for each scale value

x = assigned weight or scale value (1, 2, 3, or 4)

N = total number of responses

This measure allows the researchers to evaluate each criterion objectively and determine the level of agreement among both alpha testers (IT professionals and CCS professors) and beta testers (first-year students) with respect to CodeInsight's system quality.

4.3 Overall Mean (Grand Mean)

The grand mean is calculated to measure the performance and suitability of the CodeInsight tool for each ISO/IEC 25010:2011 quality attribute. This is calculated as an average of the weighted means of all items assessed against a particular quality attribute, such as Functional Suitability, Usability, or Performance Efficiency. The grand mean is calculated as follows:

Grand Mean = Σ Weighted Means / Number of Items

This grand mean reflects the overall performance of the system and is the basis upon which general recommendations concerning the quality of CodeInsight, as an intelligent learning aid in introductory programming courses, will be made.

Application to Evaluation Model

The statistical methods used in this research are used exclusively for the assessment of the CodeInsight system against specified quality attributes in accordance with the ISO/IEC 25010:2011 software quality model framework. This analysis concentrates on the following quality attributes:

Functional Suitability – determines whether the CodeInsight system fulfills all the desired purposes such as accurate identification of programming concept difficulty at the concept level by measuring behavioral data (error frequency, retry, and time on task), accurate calculation of the Concept Difficulty Score (CDS) through the following formula: $CDS = (0.40 \times NER) + (0.35 \times NRS) + (0.25 \times NTS)$, the ability of the AST pipeline to verify code structure, and academic integrity monitoring systems' reliability.

Usability – examines the level of ease that both students and teachers can work effectively within the system, such as how intuitive the code editor in the web application is, how understandable the dashboard and difficulty heatmap, how easy it is for users to access early warning alerts and individual student profile, and the general ease of use of the CodeInsight system. Usability will also be measured using the System Usability Scale (SUS).

Performance Efficiency – assesses the effectiveness of the system's performance when used in realistic conditions, which includes processing of real-time behavioral data after submission of the code, timely production of CDS and difficulty alerts following legitimate Submit events, and ability of the system to

operate with several users simultaneously without compromising performance efficiency.

These aspects will be evaluated based on the results from the computation of their weighted means in relation to each evaluation criterion, and their interpretations will be based on the predetermined 4-point Likert scale classifications. Such an analysis will ensure the evaluation process will adhere to internationally accepted principles in software evaluation to provide an objective basis for assessing CodeInsight's suitability for use in practice at PSU.

Pre- and Post-Assessment Analysis

Apart from the assessment using the questionnaire, this study also measures any changes in the learning progress of the students after interacting with CodeInsight by conducting pre-test and post-test evaluations. Pre-tests are performed before students engage with the system while post-tests are taken after engaging with the system for some time and with the assistance of the coding practice and difficulty profiling features available on the system. Mean scores of the two tests will be calculated and compared to identify whether using the CodeInsight system leads to any improvement in student scores on the seven basic concepts of C++ programming including Datatypes, Variables, Conditionals, Loops, Functions, Arrays, and ObjectOriented Programming.

Presentation of Data

The findings from the statistical analysis are summarized using tables to ensure that they can be interpreted clearly, systematically, and logically. Tables will be used to present figures such as frequencies, percentages, weighted means, and grand means associated with each quality attribute and criterion tested. Such a presentation technique improves comprehension and makes it easier to compare the outcome of the evaluation among various quality attributes and between both groups of respondents, alpha and beta testers.

Interpretation of Results

The computed statistical values, specifically the weighted mean and grand mean, are then interpreted based on the 4-point Likert scale that will be defined from the start. A comparison of the computed statistical value and its interpretation in relation to its corresponding numerical range gives the researcher insight into the level of agreement of the respondents when it comes to the quality of CodeInsight. Specifically, a weighted mean ranging between 3.26 to 4.00 falls under the Strongly Agree category, which means that there is a high level of acceptability. 2.51 to 3.25 is considered Agree, signifying a good level of acceptability. On the other hand, Disagree implies low acceptability and refers to 1.76 to 2.50. Finally, a strongly disagreeing attitude translates to a very low level of acceptability and refers to 1.00 to 1.75. Based on the systematic interpretation of the computed statistics, it can be determined whether the web system called CodeInsight is highly acceptable, generally acceptable, or needs more development and improvements.

Respondents of the Study

The respondents in this study are first-year students of the College of Computing Studies at Pampanga State University - Main Campus, Bacolor, Pampanga, as well as CCS professors and IT professionals with technical backgrounds. They will be selected because both groups are actual users and primarily align with the system's requirements, namely CodeInsight, a web-based system that can detect students' difficulties in programming C++ introductory courses. The respondents will be chosen based on their knowledge and relevance to the study.

Target Population

The target population of this study is first-year students taking introductory C++ courses. The population of first-year students is 550 students, which are divided

into eleven (11) sections with an estimated fifty (50) students per section. These students will be the main users of CodeInsight, as the system will be designed to monitor and support beginner programmers in their first C++ courses.

The researchers will divide respondents into two groups, namely, alpha testers and beta testers. Alpha testers consisted of CCS professors and IT professionals with technical backgrounds in system development, programming, and system evaluation. They also represent the instructor side of CodeInsight, where they monitor student progress and analytics features.

On the other hand, the beta testers will consist of first-year students taking introductory C++ courses who are expected to encounter programming difficulties when interacting with CodeInsight, which involves completing programming exercises through the system's built-in code editor and providing feedback on its learning environment. The alpha and beta testers are responsible for evaluating the system aspects of CodeInsight, including usability, functionality, and performance.

Sample Size Determination

The researchers will determine the sample size using the Slovin formula with a 10% margin of error (0.10). The total number of first-year students is estimated at 550, yielding a sample size of 85. The Slovin's formula is as follows:

$$n = \frac{N}{1 + N(e^2)}$$

Here is the computation by using Slovin's formula, by measuring the sample size:

$$\begin{aligned}n &= \frac{N}{1 + N(e^2)} \\n &= \frac{550}{1 + 550(0.10^2)} \\n &= \frac{550}{1 + 550(0.01)} \\n &= \frac{550}{1 + 5.5} \\n &= \frac{550}{6.5} \\n &\approx 84.62 \\n &\approx 85\end{aligned}$$

Where:

N = Population

n = sample size

e = Margin of Error

The total sample size is 85 respondents, all beta testers and the main respondents of the study. In contrast, alpha testers will not be included in the Slovin sample size computation and are not part of the main respondent population.

Criteria for Selection

The respondents were selected based on criteria that ensured they were all qualified to make valuable contributions to the evaluation of CodeInsight. For alpha testers, respondents should be CCS professors or IT professionals with a background in information technology, software evaluation, or programming instruction. Their expertise is especially significant, as they need experience assessing the instructor-facing aspects of CodeInsight, including the difficulty heatmap, early warning alerts, micro-concept diagnostic reports, and the academic integrity

monitoring panel, to understand the technical system evaluation and practical requirements of teaching introductory programming.

For beta testers, respondents must be first-year students currently enrolled in an introductory C++ programming course at the College of Computing Studies on the PSU Main Campus. Prior knowledge of learning analytics tools is not necessary. The main requirement is that they are enrolled in the course for which CodeInsight is intended to be deployed, since their experience with programming exercises is exactly the context that the system will design to support and analyze.

Sampling Technique

According to Wienclaw (2024), sampling is a set of techniques used to select a sample from a larger population so that a study can be conducted on a manageable group and generalized to the larger population. Which means, sampling is the process of selecting a small, specific group, called a subset, from a large population to study instead of the whole population. In this study, the researchers used sampling techniques to select students and professors as respondents to evaluate CodeInsight's functionality, usability, and performance.

Types of Sampling Techniques Used

The researchers will use non-probability sampling techniques in this study. Convenience sampling will be used to select respondents in both areas. Convenience sampling, as well as same-group non-probability sampling, refers to the availability and ease of access to a group of people when collecting data (Simkus, 2023).

The researchers will use convenience sampling to CCS professors and IT professionals, as they are the most qualified group, with technical backgrounds and an instructional side that will help the system evaluate its monitoring and analytics features.

Additionally, the researchers will also use convenience sampling to select target respondents. By using Slovin's formula, the researchers will be able to

determine 85 respondents from the 550 total population. All first-year students from each of the different sections who are readily available are allowed to participate in the evaluation.

Justification of the Sampling Technique

The study will use convenience sampling, given the nature of the study and the availability of the respondents. The researchers will employ convenience sampling of CCS professors and IT professionals with technical backgrounds to evaluate the system's functionality and features. The alpha testers will be selected based on their willingness and availability to participate during the evaluation, in which they only serve as expert evaluators for their technical knowledge and experience.

The researchers will use convenience sampling to select respondents. Since all first-year students share similar experiences and a learning environment as they are enrolled in introductory C++ courses. Hence, the researchers will select respondents based on their availability, ensuring that only those who are present and willing to participate will be included during the data collection period.

Application of the Sampling Technique

The researchers will use these sampling techniques to identify two groups of respondents and how to approach them. The CCS professors and IT professionals will serve as the alpha testers, the only qualified individuals with a technical background to assess CodeInsight's functionality, usability, and performance.

The first-year students will be the target group of the study and the beta testers will be selected by convenience sampling. Using Slovin's formula, the sample size is determined to be 85 students. The researchers will visit sections that are accessible and available during the evaluation period. Data collection will include all first-year students and a CCS professor who are present at the time of the visit and willing to use CodeInsight for evaluation.

Implementation Plan and Result

Introduction to Implementation Plan

The Implementation Plan offers a methodical process for executing the CodeInsight system, which is a Web-based application to be used in evaluating the students' source codes. The implementation phase is very important since this is the time when the integration of all the parts designed will result in the formation of a system.

Additionally, the implementation process acts as a validation process where the system's architecture, algorithm, and development process are evaluated for effectiveness within the actual working environment. This helps in ensuring that all system elements, including code parsing tools, databases, and web-based interfaces, work seamlessly and effectively together. The researchers use this opportunity to determine whether the system is capable of meeting its expected goals especially in terms of bettering code analysis and feedback processes.

Implementation Environment

The deployment of the CodeInsight system is done within a secure academic and development setting to ensure proper testing and efficient system operation. The hardware setup involves the use of normal computing tools like personal computers and laptops for students, teachers, and management. The machines have an active connection to the internet network, allowing users to access the system.

The software environment comprises a web-based program created with the use of up-to-date programming languages and frameworks and consists of a backend server along with a database management system. The system uses the methods of code analysis like the one that analyzes Abstract Syntax Tree (AST) of code. Furthermore, it uses cloud storage or local databases for storing user information, submitted code, and code analysis results. Such a combination makes sure that the system runs effectively.

Deployment Procedure

The installation process of the CodeInsight system involves a series of systematic processes, which ensure the right implementation and integration of the system.

This involves configuring the web application within the server, including the installation of necessary backend and databases, amongst other requirements. The following step would involve integration of the code analysis engine, which would be used in the analysis of the submitted codes.

Following this, the roles and permissions of the users would be set up, which would allow for the system to be accessed by the administrator, instructor, and learner, in their individual capacity.

Once the implementation process has been done, there follows the testing process in order to ascertain whether all the aspects of the system work accordingly.

User Interaction and Operation

CodeInsight is a web software product that has been created with the use of the user-centered design methodology in order to make sure that the software is friendly to its potential users. There are different kinds of users that can be accommodated by the software system and some of them include system administrators, teachers, and learners. Teachers, on the other hand, check the work submitted by learners and monitor their progress. Students, on the other hand, interact with the system by submitting their code and receiving automated analysis and feedback.

The system is accessed via a web interface that allows students to either upload or type in their codes, which are analyzed by the system analysis tool. The feedback will be produced based on the structure and accuracy of the code and the criteria for the feedback provided by the system. Some of the main functions that will be undertaken by the system will include code analysis, production of feedback, monitoring, and visualization.

Implementation Timeline

CodeInsight system implementation has a distinct timeline that has been laid down to ensure that there is systematic implementation and evaluation of the system. The first step entails the preparation of the system followed by the creation of the environment that will be used during the process. After that, there is development of the system in terms of back end and front end. Finally, there is system integration, testing, debugging, and validation of the system.

Results

This part contains the expected results that will be found after testing and evaluating the CodeInsight system by the users. This chapter aims at examining the efficiency and efficacy of the system in fulfilling its intended purpose. The anticipated results will be generated using the technical testing process and responses from the questionnaire distributed among the respondents.

Results of System Testing

From the results of system testing, it can be expected that the CodeInsight system functions can analyze and evaluate the submitted codes from the students.

Firstly, the processing of code inputs by the system through its analysis engine is expected to be successful in recognizing the syntax error, logical problems, and other inconsistencies of codes. Through this implementation, the AST analysis helps improve code evaluation significantly.

Secondly, the performance of the system in testing is expected to be reliable because there will be a seamless process in which the system evaluates the codes submitted. The interaction between the frontend and backend is also expected to be successful and instant.

Lastly, the system is expected to be reliable even when continuously used. The researchers expect no failures will be encountered in using this system.

Evaluation Result

The evaluation process of the CodeInsight system is intended to be carried out through a structured questionnaire, based on the ISO/IEC 25010 software quality model. This will measure some critical attributes of software, including functionality, usability, reliability, performance efficiency, and security.

The anticipated result of the evaluation process is that both alpha and beta testers will give favorable evaluations of the software. Alpha testers (instructors of IT and technical evaluators) will determine the technical correctness and adherence to programming evaluation standards of the system. However the beta testers, who are first-year programming students, will notice that the program is helpful for spotting mistakes in code as well as understanding programming concepts. The weighted mean approach will be adopted to analyze the survey results after the data collection process.

Presentation of Results

Results will be display through the use of tables containing frequency distribution, percentage values, and weighted mean score for each criterion will be expected to be used in evaluating the system.

Through tables, one is able to organize the data collected such that comparisons between responses from various users become easier. The correct labeling and organization of data in tables is expected to enhance clarity.

The structured presentation will expect to help stakeholders quickly understand the system's performance and evaluation outcomes.

Interpretation of Results

Based on the development and evaluation of the system, it is expected that the software will effectively assist in the analysis of the codes and determination of any problems in programming conceptually.

Given the high degree of expected effectiveness in terms of usability and functionality, it can be said that the software is able to analyze the input entered by the user and give pertinent feedback. Moreover, the software is expected to be reliable through consistent analysis of the inputs given.

However, there are some limitations of the software, such as the need for stable internet connectivity and less efficiency in analyzing highly complex codes.

Despite the mentioned limitations, the software is expected to be an effective teaching tool for both the students and their teachers.

Ethical Considerations

The ethical guidelines with regard to research on human subjects will be adhered to in this research through proper handling of the subjects during data collection, system usage, and data analysis stages.

The consent of the participants will be acquired before undertaking the experiment. The participants will include both the students and their teachers enrolled in the introduction to C++ programming class offered in the Pampanga State University. Before acquiring their consent, they will be made aware of the goals of the experiment, nature of the data to be collected, and purposes for which the data will be used to enable them to willingly participate in this experiment. Participation in the experiment is completely voluntary as they may decide not to participate at any stage of the experiment.

The behavioral coding data collected from the experiment, which includes the number of mistakes committed, number of trials, and completion time, will only be used for academic research purposes. Besides, information related to each participant will be stored solely within the university's network infrastructure throughout the data analysis phase.

The additional level of confidentiality regarding participants' privacy includes that only the participating instructors and researchers have access to the individual student's information and their profile regarding the degree of difficulty. Information about individuals is not to be revealed in the final report. On the other hand, data will be presented in aggregate form or anonymous form in the data findings.

Concerning the issue of academic honesty in CodeInsight, there will be mechanisms that verify and detect the case where coding is hardcoded, template-only, and invalid algorithmically.

References

- Ahadi, A., Lister, R., & Vihavainen, A. (2016). On the number of attempts students made on some online programming exercises during semester and their subsequent performance on final exam questions. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education* (pp. 218–223). Association for Computing Machinery. <https://doi.org/10.1145/2899415.2899452>
- Altadmri, A., & Brown, N. C. (2015). 37 million compilations: Investigating novice programming mistakes in large-scale student data. In *Proceedings of the 46th ACM technical symposium on computer science education* (pp. 522–527). ACM. <https://doi.org/10.1145/2676723.2677258>
- Bennedsen, J., & Caspersen, M. E. (2019). Failure rates in introductory programming: 12 years later. *ACM Inroads*, 10(2), 30–36. <https://doi.org/10.1145/3324888>
- Alonzo, A. C. (2022). Learning progressions as a simplified model: Examining teachers' reported uses to inform classroom assessment practices. *Science Education*, 106(4), 922–958. <https://doi.org/10.1002/sce.21713>
- Alvarez, C., Samary, M. M., & Wise, A. F. (2023). Modularization for mastery learning in CS1: A 4-year action research study. *Journal of Computing in Higher Education*, 36(2), 546–589. <https://doi.org/10.1007/s12528-023-09366-1>
- Banihashem, S. K., Gašević, D., Noroozi, O., Jarodzka, H., Brinke, D. J., & Drachsler, H. (2025). Optimizing formative assessment with learning analytics. *Review of Educational Research*. <https://doi.org/10.3102/00346543251370753>
- Barczak, A. L. C., Mathrani, A., Han, B., & Reyes, N. H. (2023). Automated assessment system for programming courses: A case study for teaching data structures and algorithms. *Educational Technology Research and Development*, 71(6), 2365–2388. <https://doi.org/10.1007/s11423-023-10277-2>
- Batten, J. (2025, August). Infographic: Computing bachelor's enrollment continues to grow even as the field evolves. *Computing Research News*. <https://cra.org/crn/2025/08/infographic-computing-bachelors-enrollment-continues-to-grow-even-as-the-field-evolves/>

Bibangco, I., & Dionson, A. (2024). Identifying at-risk students in programming courses: A Philippine perspective. *Journal of Information Technology Education: Research*, 23, 145–158. <https://doi.org/10.28945/5038>

Bigotte de Almeida, M. E., et al. (2023, May). *Failure rates in Maths and CS 1: Are they associated?* ResearchGate. https://www.researchgate.net/publication/370975931_Failure_rates_in_Maths_and_CS_1_are_they_associated

Chung, C.-Y., Hsiao, I.-H., & Lin, Y.-L. (2022). AI-assisted programming question generation using knowledge graph and abstract syntax tree. *Journal of Research on Technology in Education*. <https://doi.org/10.1080/15391523.2022.2123872>

Dela Rosa, M. (2023). Learning management systems in Philippine higher education: Current state and future directions. *Technology-Enhanced Learning in the Asia-Pacific Region*, 45(3), 234–249. <https://doi.org/10.1007/978-981-99-0123-456>

Gao, Z., Yan, H., Liu, J., Zhang, X., Lin, Y., Zhang, Y., Sun, X., & Feng, J. (2025). Tracing distinct learning trajectories in introductory programming course: A sequence analysis of score, engagement, and code metrics for novice computer science vs. math cohorts. *International Journal of STEM Education*, 12(1), 1–26. <https://doi.org/10.1186/s40594-025-00546-2>

Gregorczyk, H. (2023). Transparency and proportionality: A two-step approach to assessing fairness in the “Privacy Act.” *Monash University Law Review*, 49(3), 236–260. <https://search.informit.org/doi/10.3316/informit.T2024120900016191789976218>

Haldeman, G., Babes-Vroman, M., Tjang, A., & Nguyen, T. D. (2021). CSF: Formative feedback in autograding. *ACM Transactions on Computing Education*, 21(3), Article 21. <https://eric.ed.gov/?id=EJ1314295&utm>

Hsu, W.-C., & Gainsburg, J. (2021). Hybrid and non-hybrid block-based programming languages in an introductory college computer science course. *Journal*

of Educational Computing Research, 59(5).
<https://doi.org/10.1177/0735633120985108>

Karaman, P. (2021). The effect of formative assessment practices on student learning: A meta-analysis study. *International Journal of Assessment Tools in Education*, 8(4), 801–817. <https://doi.org/10.21449/ijate.870300>

Leinonen, J., Castro, F. E. V., & Hellas, A. (2022). Time-on-task metrics for predicting performance. *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education*, 1, 871–877. <https://doi.org/10.1145/3478431.3499359>

López-Pernas, S., Saqr, M., & Viberg, O. (2021). Putting it all together: Combining learning analytics methods and data sources to understand students' approaches to learning programming. *Sustainability*, 13(9), 4825. <https://doi.org/10.3390/su13094825>

Mai, T. T., Bezbradica, M., & Crane, M. (2021). Learning behaviours data in programming education: Community analysis and outcome prediction with cleaned data. *Future Generation Computer Systems*, 127, 42–55. <https://doi.org/10.1016/j.future.2021.08.026>

Mai, T. T., Crane, M., & Bezbradica, M. (2023). Students' learning behaviour in programming education analysis: Insights from entropy and community detection. *Entropy*, 25(8), 1225. <https://doi.org/10.3390/e25081225>

Maranga, X. D., Pausanos, M. V., Sinoy, S., Romana, C. L. S., & Arellano, C. (n.d.). Integrating adaptive assessment in CodeChum for personalized programming education. *Proceedings of the 2024 10th International Conference on Education and Training Technologies (ICETT '24)*, 89–95. <https://doi.org/10.1145/3661904.3661910>

Namoun, A., & Alshankiti, A. (2021). Predicting student performance in an online learning environment using learning analytics. *Education and Information Technologies*, 26(2), 1533–1553. <https://doi.org/10.1007/s10639-020-10341-z>

National Academies of Sciences, Engineering, and Medicine. (2021). *Building teaching capacity in computer science*. The National Academies Press. <https://doi.org/10.17226/26189>

Nguyen, A.-T. P., & Hoang, V.-D. (2024). Development of code evaluation system based on abstract syntax tree. *Journal of Technical Education Science*. <https://jte.edu.vn/index.php/jte/article/view/1514>

Omer, U., Rabia Tehseen, Farooq, M. S., & Abid, A. (2023). Learning analytics in programming courses: Review and implications. *Education and Information Technologies*, 28(9), 11221–11268. <https://doi.org/10.1007/s10639-023-11611-0>

Pacaci, C., Yalcin, S. A., & Yalcin, N. (2024). Effectiveness of conceptual change strategies in science education: A meta-analysis. *Journal of Research in Science Teaching*, 61(3), 648–685. <https://doi.org/10.1002/tea.21887>

Pak, J., Lee, J., & Lee, M. (2022). Developing a learning data collection platform for learning analytics in online education. *Applied Sciences*, 12(11), 5412. <https://doi.org/10.3390/app12115412>

Paolucci, M., Sapuppo, F., & Lodi, M. (2024). Learning analytics in computer science education: Promises and challenges. *Computers & Education*, 206, 104895. <https://doi.org/10.1016/j.compedu.2023.104895>

Pires, J. P. J., Brito Correia, F., Gomes, A., Borges, A. R., & Bernardino, J. (2024). Predicting student performance in introductory programming courses. *Computers*, 13(9), 219. <https://doi.org/10.3390/computers13090219>

Rahimi, Z., Grace, K., Hauff, C., & Pammer-Schindler, V. (2025). Learning analytics for introductory programming: A systematic review. *IEEE Transactions on Learning Technologies*, 18(1), 45–62. <https://doi.org/10.1109/TLT.2024.3456789>

Rodríguez-Mora, F., Pontes-Pedrajas, A., & Poyato-López, F. J. (2024). Misconceptions in the learning of natural sciences: A systematic review. *Education Sciences*, 14(5), 497. <https://doi.org/10.3390/educsci14050497>

Rouben, J., Carola, A., Seong-Ho, L., & Ratimir, T. (2023). Investigating the impact of real-time feedback formative assessment on high school algebra learning outcomes in the United States. *Research and Advances in Education*, 2(12), 7–16. <https://doi.org/10.56397/RAE.2024.09.05>

Senger, A., Edwards, S. H., & Ellis, M. (2022). Helping student programmers through industrial-strength static analysis: A replication study. In *SIGCSE 2022: Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1* (pp. 8–14). Association for Computing Machinery. <https://dl.acm.org/doi/10.1145/3478431.3499310#bibliography>

Simkus, J. (2023, July 31). *Convenience sampling (accidental sampling): Definition, method & examples*. Simply Psychology. <https://www.simplypsychology.org/convenience-sampling.html>

Sortwell, A., Trimble, K., Ferraz, R., Geelan, D. R., Hine, G., Ramirez-Campillo, R., Carter-Thuiller, B., Gkintoni, E., & Xuan, Q. (2024). A systematic review of meta-analyses on the impact of formative assessment on K-12 students' learning: Toward sustainable quality education. *Sustainability*, 16(17), 7826. <https://doi.org/10.3390/su16177826>

Sun, W., Fang, C., Miao, Y., et al. (2023). *Abstract syntax tree for programming language understanding and representation*. <https://www.catalyzex.com/paper/abstract-syntax-tree-for-programming-language>

uniRank. (2025). *Central Luzon higher education institution rankings*. Retrieved from <https://www.webometrics.info/>

Utamachant, P., Anutariya, C., & Pongnumkul, S. (2023). i-Ntervene: Applying an evidence-based learning analytics intervention to support computer programming instruction. *Smart Learning Environments*, 10(1). <https://doi.org/10.1186/s40561-023-00257-7>

Wienclaw, A. (2024). *Sampling | Social sciences and humanities | Research starters*. EBSCO.

<https://www.ebsco.com/research-starters/socialsciences-and-humanities/sampling>

Yao, D., & Lin, J. (2025). Cognitive enhancement through competency-based programming education: A 12-year longitudinal study. *Education and Information Technologies*. <https://doi.org/10.1007/s10639-025-13582-w>